

山东大学（青岛） · 计算机科学与技术学院

全国大学生计算机系统能力大赛

RespOS

操作系统内核设计文档

Rust · RISC-V 64 · LoongArch 64 · Linux ABI Compatibility

比特工匠队

决赛设计文档 · 2026 年

目 录

1. 概述	7
1.1 项目定位与设计目标	7
1.2 从启动到用户程序的整体架构	7
1.3 核心对象与模块边界	8
1.4 贯穿各模块的设计方法	9
1.4.1 用对象模型表达 Linux 语义	9
1.4.2 以生命周期和提交边界保证失败安全	9
1.4.3 把架构差异收敛在硬件边界	10
1.5 各模块的功能主线与章节导航	10
1.6 功能与设计总结	11
2. 进程管理	13
2.1 概述与设计目标	13
2.2 统一任务对象与资源所有权	13
2.2.1 TaskControlBlock	13
2.2.2 强弱引用与生命周期	14
2.3 任务状态与生命周期	15
2.3.1 状态机与调度不变量	15
2.3.2 创建: fork、clone 与共享边界	16
2.3.3 exec: 先准备、再静止、后提交	17
2.3.4 exit、wait 与延迟析构	18
2.4 分层优先级调度	19
2.5 上下文切换与内核栈安全	19
2.6 多核处理器管理	20
2.7 跨模块协作与功能总结	21
3. 中断与异常处理	22
3.1 模块定位	22
3.2 Trap 的基本流程	22
3.3 TrapContext: 统一的可恢复现场	23
3.3.1 RISC-V64 的 TrapContext	23
3.3.2 LoongArch64 的 TrapContext	23
3.3.3 两种上下文共同语义	24
3.4 中断屏蔽与并发安全	24
3.5 功能与设计总结	25
4. 内存管理	26
4.1 分层对象与核心不变量	26
4.2 双架构地址空间布局	27
4.3 物理页帧与内核堆	28

4.3.1 RV64 动态物理内存发现	28
4.4 VMA 变换与映射类型	29
4.5 缺页处理与按需分配	29
4.6 写时复制	30
4.7 共享文件映射与锁外写回	30
4.8 大 ELF 按需装载	31
4.9 多核 TLB 可见性与生命周期	32
4.10 与其他模块的协作	32
4.11 功能与设计成果总结	33
5. 文件系统	34
5.1 VFS 对象模型	34
5.1.1 File: 一次打开文件的运行时状态	34
5.1.2 FileOp: 统一不同类型文件的使用方式	35
5.1.3 InodeOp: 文件对象的后端能力边界	35
5.1.4 Ext4Inode: InodeOp 的具体实现	36
5.1.5 Dentry、Path 与对象定位	37
5.2 挂载树与路径解析	37
5.2.1 VfsMount 与 Mount	37
5.2.2 namei 路径遍历	38
5.3 ext4 与虚拟文件系统后端	39
5.3.1 ext4 适配层	39
5.3.2 procfs、devfs 与特殊文件	39
5.4 页缓存与持久化写回	39
5.4.1 PageCache 的职责	39
5.4.2 三层写回链	39
5.4.3 与 mmap(MAP_SHARED) 的协作	40
5.5 文件描述符表与生命周期	40
5.5.1 FdTable 与 FdEntry	40
5.5.2 dup、fork、exec 与 close	40
5.6 功能与设计总结	41
6. 进程间通信与信号	42
6.1 概述与机制分工	42
6.2 信号机制	42
6.2.1 线程状态与进程共享状态	42
6.2.2 从产生到 sigreturn 的完整链路	43
6.2.3 可中断等待与架构安全约束	44
6.3 管道: 字节流、阻塞与端点生命周期	44
6.3.1 核心对象	44
6.3.2 读写阻塞协议	45

6.3.3 EOF、EPIPE 与 poll 可见性	45
6.4 Futex: 用户态快路径与内核竞争路径	46
6.4.1 等待对象与 key	46
6.4.2 single-winner 完成协议	46
6.4.3 requeue 的线性化与当前边界	47
6.5 System V 共享内存	47
6.5.1 段对象与映射生命周期	47
6.5.2 shmat 的失败原子性	48
6.6 跨模块协作与功能总结	48
7. 时钟模块	50
7.1 时间模型与核心数据	50
7.1.1 三种时间尺度	50
7.1.2 TimeSpec 与 deadline	50
7.2 时钟源与时钟中断	51
7.2.1 RISC-V 与 LoongArch 的时钟源	51
7.2.2 Timer trap 的职责	51
7.3 超时管理与任务唤醒	52
7.3.1 deadline registry	52
7.3.2 nanosleep 与阻塞唤醒	52
7.3.3 interval timer 与 POSIX timer	53
7.4 用户态时间接口	53
7.4.1 clock_gettime 与系统时间	53
7.4.2 timerfd: 将时间转换为文件事件	53
7.5 与其他模块的协作	54
8. 网络模块	55
8.1 分层架构与责任边界	55
8.2 Socket 对象与 fd 生命周期	56
8.3 回环设备与协议轮询	57
8.4 TCP 套接字与并发监听	57
8.4.1 阻塞、非阻塞与信号中断	58
8.4.2 backlog listener 池	58
8.5 UDP 与 UNIX 域 socket	59
8.5.1 UDP 数据报	59
8.5.2 UNIX 域 socket	60
8.6 网络系统调用与就绪性	60
8.6.1 回环 IPv6 兼容边界	61
8.7 与其他模块的协作	61
8.8 功能与设计成果总结	61
9. 硬件抽象层与架构适配	63

9.1 架构适配的整体结构	63
9.1.1 一套公共调用链，两套底层实现	63
9.1.2 双架构差异总览	64
9.2 地址空间与页表适配	64
9.2.1 共享高半区内核模型	64
9.2.2 RISC-V Sv39 页表	64
9.2.3 LoongArch 页表与 TLB refill	64
9.2.4 页表操作的统一语义	65
9.3 Trap 与特权级切换	65
9.3.1 统一的 trap 三阶段	65
9.3.2 TrapContext 的统一字段与架构差异	65
9.3.3 返回用户态的安全顺序	66
9.4 任务上下文与调度切换	66
9.4.1 TaskContext 与 TrapContext 的分工	66
9.4.2 两套 switch.S 的共同布局	66
9.5 启动流程适配	66
9.5.1 公共启动阶段	66
9.5.2 RISC-V 启动路径	67
9.5.3 LoongArch 启动路径	67
9.6 功能与设计总结	67
10. 设备驱动	69
10.1 概述	69
10.2 块设备驱动	69
10.3 网卡驱动	69
10.4 串口驱动	70
10.5 设备树与设备发现	70
11. 项目参考、AI 使用与第三方代码	71
11.1 项目参考与相关对比	71
11.2 AI 工具使用说明	71
11.3 AI 产生内容与人工复核案例	72
11.4 第三方代码与依赖	72
11.5 资料引用与许可证说明	72
12. 总结与展望	73
12.1 本队最终贡献摘要	73
12.2 当前限制与未完成项	73
12.3 测试结果与性能数据	74
12.4 复现入口	74
12.5 未来工作	74

1. 概述

本章回答: RespOS 面向什么目标构建? 各内核模块如何协作? 双架构、Linux ABI 兼容和 Rust 工程实践如何共同落到可运行的系统上?

1.1 项目定位与设计目标

RespOS 是一个使用 Rust 编写的教学与竞赛型宏内核, 面向全国大学生操作系统比赛的用户态工作负载, 支持 RISC-V64 与 LoongArch64 两种架构。项目的主要目标不是实现一组彼此独立的系统调用, 而是构建一条能够运行真实用户程序的内核执行链: 从架构启动、地址空间和任务调度, 到文件、网络、信号、时钟和设备, 再通过接近 Linux 的 ABI 向用户态提供服务。

内核设计围绕三个相互约束的目标展开:

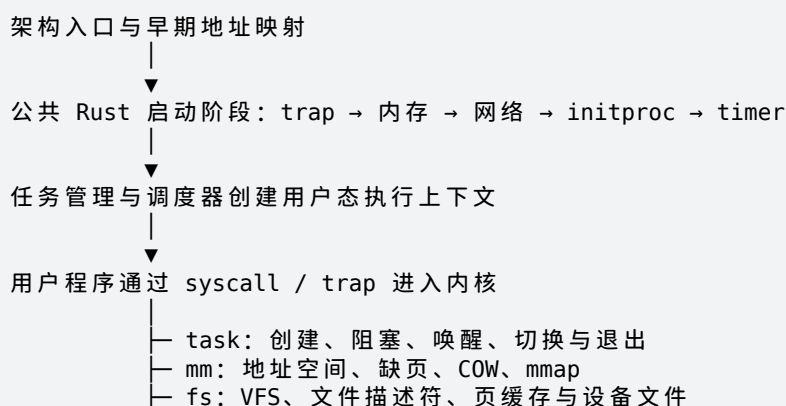
设计目标	RespOS 的实现方向	评委可观察的结果
跨架构复用	公共模块复用任务、内存、文件和系统调用逻辑; 启动、页表、trap、切换和计时在 arch/ 中特化	同一套内核功能可以落到 RV64 与 LA64 的不同特权体系
Linux 用户态兼容	系统调用层负责 ABI 参数、返回值和错误码; 领域模块维护实际对象与状态机	musl/glibc 程序可以沿统一 syscall 边界运行
工程上的可维护性	Rust 所有权、锁和 RAII 管理资源生命周期; 失败路径先准备、后提交	fork、exec、mmap、mount、socket、信号等复杂操作具有明确的提交边界

表 1-1 RespOS 的总体设计目标

这些目标决定了后续章节的叙述重点。常规的 Linux 接口只作必要说明; 真正展开的是项目自身的数据结构、跨模块调用链、并发协议、资源生命周期和架构适配, 因为这些部分体现了内核设计工作的主要内容。

1.2 从启动到用户程序的整体架构

RespOS 采用宏内核结构: 内核子系统运行在同一特权级地址空间中, 通过明确的模块边界组织职责; 用户程序则通过系统调用、信号和文件/网络接口与内核交互。一次典型的启动与执行链如下:



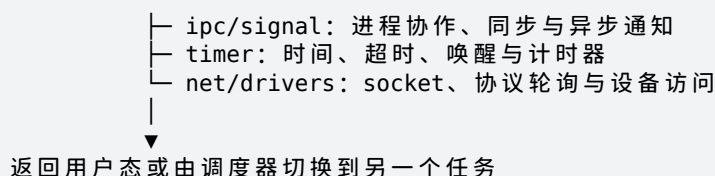


图 1-1 RespOS 从启动到用户态执行的整体调用链

公共启动顺序体现了模块之间的依赖关系: trap 入口必须先可用, 内存管理必须在创建任务和网络对象前完成, initproc 入队后才进入正式调度, timer interrupt 则在调度路径准备好之后开启。用户程序运行后, 系统调用和硬件 trap 共同成为进入内核的入口, 最终由任务管理决定继续返回当前任务还是切换到其他任务。

从代码组织看, 内核主要由以下几类部分组成:

```

os/src/
├── arch/      # RV64/LA64 启动、页表、trap、上下文、时钟
├── task/      # TCB、线程组、调度器、futex、退出回收
├── mm/        # 页帧、堆、MemorySet、VMA、COW、文件映射
├── fs/        # VFS、ext4、mount、namei、fd、page cache、pipe
├── signal/    # pending、handler、siginfo、alt stack、sigreturn
├── net/       # socket、TCP/UDP、UNIX socket、loopback、监听池
├── drivers/   # virtio block/net、串口与设备中断
├── syscall/   # Linux ABI 参数解析和各领域入口
└── main.rs    # 公共启动入口
  
```

图 1-2 RespOS 内核源码的主要模块

这里的目录划分并不等同于完全独立的服务。各模块通过对象引用、任务状态和系统调用边界协作。例如, 管道阻塞需要 task 的等待队列和 signal 的可中断语义, mmap 文件映射同时依赖 MM 的 VMA/PTE 和 FS 的 FileOp/页缓存, 网络 socket 则通过 FS 的 FileOp 进入统一 fd 生命周期。

1.3 核心对象与模块边界

RespOS 把“执行单元”“地址空间”“打开文件”和“硬件上下文”分开建模, 避免一个对象同时承担过多身份。表 1-2 给出了贯穿多个章节的核心对象。

对象	所属模块	负责的状态	与其他模块的连接
TaskControlBlock	task	tid/tgid、任务状态、内核栈、资源引用、调度属性	持有 MemorySet、FdTable、信号状态，并进入 scheduler
MemorySet / VMA	mm	用户地址空间、映射权限、驻留页、文件 backing	trap 缺页入口调用，exec/mmap/clone 修改
TrapContext	arch/trap	用户寄存器、返回 PC、特权状态	syscall、signal、page fault 修改后由返回汇编恢复
TaskContext	task/arch	内核切换点、内核栈和地址空间 token	__switch 保存与恢复，连接 scheduler 和页表
FdTable / FileOp	fs	描述符旗标、open-file 状态、偏移和对象操作	pipe、socket、timerfd、设备文件复用 fd 接口
Dentry / InodeOp	fs	路径身份和后端文件对象	namei、mount、ext4、procfs、devfs 共享 VFS 入口
timeout registry	timer/task/signal	deadline、等待者和到期动作	唤醒 nanosleep/futex，投递信号，通知 timerfd

表 1-2 RespOS 的核心对象与跨模块连接

这些对象之间存在清晰的所有权和生命周期边界。TCB 是调度器真正管理的执行单元，但用户态现场保存在其内核栈上的 TrapContext 中；TaskContext 只保存内核态切换所需的最小状态。FdTable 保存描述符层关系，真正的 open-file 状态由共享的 FileOp 持有，因此 dup、fork 和线程创建可以分别表达“共享打开实例”和“是否共享描述符表”。类似地，VMA 描述地址范围和权限，实际页帧由页表和映射 backing 管理，缺页时再按需建立 PTE。

1.4 贯穿各模块的设计方法

1.4.1 用对象模型表达 Linux 语义

RespOS 没有把 Linux ABI 直接实现成大量 syscall 分支，而是将用户可观察语义落到稳定对象上：进程和线程统一由 TCB 表示，fork/clone 通过资源共享边界表达差异；文件描述符、open-file description、路径、dentry 和 inode 分层表达文件语义；socket、pipe 和 timerfd 则作为 FileOp 接入同一套 fd 生命周期。

这种分层使系统调用层保持较薄。syscall 负责从用户空间读取参数、检查范围和权限、调用领域对象，并把结果转换为 ABI 规定的返回值；真正的状态转换、阻塞唤醒和失败回滚由 task、mm、fs、net 等模块各自维护。

1.4.2 以生命周期和提交边界保证失败安全

内核中许多操作都不是单次赋值，而是跨越多个对象的状态迁移。RespOS 对这类操作采用“先准备、再提交、最后释放旧状态”的原则：

操作	先准备的内容	提交时机	需要保持的关系
fork/clone	子任务、地址空间元数据、共享资源引用	子任务状态和资源关系准备完成后入队	父子任务不能重复入队, COW 权限修改保持一致
exec	ELF、VMA、用户栈和参数辅助向量	新映像可运行后替换旧地址空间	旧映像失败时仍可继续运行, 其他线程先完成静止协议
mmap/缺页	VMA、页帧或文件 backing	权限和资源准备成功后安装 PTE	VMA、PTE 和页帧状态相容
mount/namei	mount、dentry 和目标路径	挂载关系验证后接入挂载树	路径身份与后端 inode 不混淆
socket/accept	协议对象、listener handle 和 fd	对象所有权明确后发布 fd	handle 不重复移交, 失败时恢复队列和状态

表 1-3 典型操作的准备—提交边界

生命周期协议同样贯穿阻塞和回收: 任务进入 `blocked` 集合后才发布可能唤醒它的事件, `futex/pipe/socket` 的竞争路径使用单赢家完成状态, 退出任务则延迟到安全栈上析构。这样的实现重点不是让所有路径看起来相同, 而是让每个跨模块操作都能说明“谁拥有对象、谁可以提交、失败后如何恢复”。

1.4.3 把架构差异收敛在硬件边界

RISC-V64 和 LoongArch64 不共享寄存器、页表根寄存器或异常返回指令, 因此 RespOS 没有强行把底层代码写成完全相同的实现。公共代码通过 `arch/mod.rs` 的条件编译和模块重导出使用统一语义入口; 架构目录分别实现启动、地址转换、`trap entry/return`、任务切换、时钟和中断控制。

例如, 公共 `trap handler` 只需要通过 `get_a0`、`set_a0`、`get_sepc` 等方法读写上下文, 具体是 RISC-V 的 `x[10]/sepc`, 还是 LoongArch 的 `x[4]/era`, 由架构上下文实现决定。公共内存管理也只依赖页表的 `map`、`unmap`、`query` 和刷新语义, 而不直接散落 `satp`、`PGDL/PGDH` 或 `TLB` 指令。

1.5 各模块的功能主线与章节导航

第 2—9 章按照“执行基础 → 资源管理 → 外部交互 → 硬件落地”的顺序展开。各章并非简单罗列接口, 而是分别解释一个关键内核问题:

章节	核心问题	主要设计内容
第 2 章 进程管理	任务如何创建、调度、阻塞、退出并在多核下安全切换?	TCB、线程组、clone/exec、调度队列、上下文切换、SMP 生命周期
第 3 章 中断与异常处理	trap 如何保存现场、分发事件并安全返回?	TrapContext、syscall、缺页、timer trap、信号交付、双架构返回协议
第 4 章 内存管理	地址空间如何表示, 页如何按需建立和回收?	VMA/PTE/frame、lazy allocation、COW、文件映射、TLB 可见性
第 5 章 文件系统	fd、路径和后端文件系统如何连接?	VFS、mount/namei、ext4、procfs/devfs、页缓存、fd 生命周期
第 6 章 进程间通信与信号	任务如何通知、传输、同步和共享内存?	signal、pipe、futex、System V 共享内存及其竞争协议
第 7 章 时钟模块	硬件计数器如何驱动超时、唤醒和用户时间接口?	时间尺度、deadline registry、sleep、interval/POSIX timer、timerfd
第 8 章 网络模块	socket ABI 如何连接协议栈、设备和阻塞模型?	smoltcp、TCP/UDP/UNIX socket、loopback、listener pool、poll
第 9 章 硬件抽象层与架构适配	同一套公共内核如何落到两种指令集?	启动、页表、trap、TaskContext、时钟和架构边界

表 1-4 第 2—9 章的功能主线

阅读这些章节时, 可以沿着三条贯穿全书的链路理解 RespOS。第一条是执行链: 启动建立硬件环境, task 创建和调度执行单元, trap 在用户态与内核态之间切换。第二条是资源链: mm 管理地址空间, fs 管理文件和设备对象, net 将 socket 接入 fd 生命周期。第三条是事件链: timer 产生时间事件, signal、pipe、futex 和 socket 将事件转换为唤醒、通知或数据可见性。

1.6 功能与设计总结

RespOS 的整体设计可以概括为表 1-5:

设计成果	形成方式	对系统的作用
双架构公共内核	公共 Rust 模块 + arch/ 内的架构特化入口	保持任务、内存、文件和网络语义一致, 同时适配两套硬件体系
统一执行模型	TCB、TrapContext、TaskContext 和 scheduler 分工	将用户态运行、内核态处理和任务切换连接为完整闭环
统一资源接口	VMA、VFS、FileOp、socket 和 timer registry 分层	让 mmap、文件、网络、设备和计时器能够复用成熟的生命周期机制
Linux ABI 兼容路径	syscall 薄层 + 领域模块状态机	使真实用户程序通过一致的参数、返回值和错误语义使用内核
面向并发的工程实现	锁、原子状态、single-winner 协议和延迟析构	覆盖多核调度、阻塞唤醒、页表修改和资源回收中的竞态

表 1-5 RespOS 的总体设计成果

后续章节将从执行单元开始，依次展开任务、trap、内存、文件、IPC、时钟、网络和架构适配。它们共同说明 RespOS 如何把一个面向比赛测例的内核目标，落实为可启动、可调度、可交互并能够在两种架构上复用的系统实现。

2. 进程管理

本章回答：RespOS 如何统一表示进程与线程，并在创建、调度、上下文切换、退出和多核并发之间维持一致的任务生命周期？

2.1 概述与设计目标

RespOS 采用与 Linux task 模型相近的统一任务设计：内核只维护一种可调度对象 `TaskControlBlock` (TCB)，进程与线程的差异由身份和资源共享边界表达，而不是拆成两套控制块。每个任务都有独立的 `tid`、内核栈、寄存器现场和调度状态；线程组组长满足 `tid == tgid`，同组线程拥有不同 `tid`，但共享 `tgid`。因此，`gettid()` 标识具体执行线程，`getpid()` 返回线程组身份，调度器始终按 `tid` 调度，进程级退出、信号和等待则以线程组或 `tgid` 为边界。

这一模型要同时解决五类问题：任务对象如何持有资源，`clone` 如何组合共享语义，阻塞与唤醒如何避免丢失事件，任务切换如何保证旧上下文已经保存，以及退出任务如何在不释放当前内核栈的前提下完成回收。表 2-1 给出了这些问题与 RespOS 设计的对应关系。

表 2-1 进程管理的设计目标与实现抓手

设计目标	RespOS 的实现	产生的效果
统一进程与线程	TCB + <code>tid/tgid</code> + <code>ThreadGroup</code>	创建、调度、阻塞、信号和退出复用同一套任务接口
明确资源共享边界	<code>Arc<MemorySet></code> 、 <code>Arc<FdTable></code> 、共享 handler 表	<code>clone</code> 可按 <code>flag</code> 选择共享或复制， <code>fork/</code> 线程语义不混用
保证阻塞唤醒正确	<code>ready/blocked</code> 分离、事件单赢家、发布后复查	覆盖 <code>wait</code> 、 <code>futex</code> 、 <code>pipe</code> 和 <code>signal</code> 的 <code>lost-wakeup</code> 窗口
保证多核切换安全	<code>per-CPU idle context</code> 、 <code>handoff slot</code> 、 <code>cpu_owner</code>	其他 CPU 不会恢复尚未保存完成的任务上下文
分离退出与析构	<code>Exited</code> 状态、父进程强引用、 <code>DEAD_TASKS</code>	<code>wait status</code> 可保留，TCB 与内核栈在安全栈上延迟释放

2.2 统一任务对象与资源所有权

2.2.1 TaskControlBlock

TCB 不是简单的“寄存器集合”，而是一个任务从创建到析构的资源所有权中心。代码片段 2-1 摘录了当前实现中最能说明身份、调度、进程关系、资源共享和信号状态的字段；未列出的 `UID/GID`、调度属性、资源限制和计时字段遵循相同的所有权原则。

代码片段 2-1 `TaskControlBlock` 的代表性字段

```
// os/src/task/task.rs (节选)
pub struct TaskControlBlock {
    kernel_stack: KernelStack,
    tid: RwLock<TidHandle>,
```

```

    tgid: AtomicUsize,

    thread_group: Arc<SpinLock<ThreadGroup>>,
    group_exiting: Arc<AtomicBool>,
    terminate_requested: AtomicBool,
    task_status: SpinLock<TaskStatus>,
    cpu_owner: AtomicUsize,

    parent: Arc<SpinLock<Option<Weak<TaskControlBlock>>>>,
    children: Arc<SpinLock<BTreeMap<usize, Arc<TaskControlBlock>>>>,
    exited_children: Arc<SpinLock<BTreeSet<usize>>>>,

    memory_set: Arc<RwLock<MemorySet>>,
    fd_table: SpinLock<Arc<FdTable>>,

    sig_pending: SpinLock<SigPending>,
    sig_stack: SpinLock<SignalStack>,
    sig_handler: Arc<SpinLock<SigHandler>>,

    interruptible: AtomicBool,
    waiting_for_child: AtomicBool,
    interrupted: AtomicBool,
}

```

代码片段 2-1 体现了三种并发手段的分工。原子字段用于频繁读取且状态较小的身份或协议位，例如 `tgid`、终止请求和 CPU owner；`SpinLock` 保护需要整体一致更新的任务状态、父子集合和信号状态；`RwLock<MemorySet>` 允许地址空间查询并发进行，同时把页表或 VMA 修改收敛到写侧。`trap context` 并不是 TCB 中的独立字段，而是固定放在该任务内核栈顶端；`TaskContext` 位于其下方，由 `__switch` 保存和恢复。

2.2.2 强弱引用与生命周期

RespOS 根据对象是否应延长任务生命周期选择 `Arc` 或 `Weak`。如表 2-2 所示，线程组和全局任务索引只保存 TCB 的弱引用，避免“索引持有对象、对象又持有索引关系”造成无法析构；父任务的 `children` 则有意持有子进程 leader 的强引用，使子进程退出后仍能保留 wait status，直到 `wait4` 提交回收。

表 2-2 任务相关对象的所有权与共享范围

对象或状态	创建者与共享范围	更新/释放方式	用户可见语义
KernelStack、trap context、TaskContext	每个 TCB 独占	CPU 切到 idle 栈后随 TCB 延迟析构；slot 可复用	每个线程可独立陷入内核和被调度
ThreadGroup	同组线程共享，成员表保存 Weak<TCB>	创建线程时加入，线程退出或 exec 清理时移除	tid 不同但 tgid 相同，组级操作作用于全部线程
MemorySet	fork 默认建立 COW 子空间；CLONE_VM 可共享	exec 替换，最后一个所有者退出时回收	决定进程间内存隔离或线程间地址空间共享
FdTable	fork 复制描述符表；CLONE_FILES 或线程创建时共享	close/exec/exit 更新；exec 先私有化再处理 CLOEXEC	决定一方 close 是否影响另一方的 fd 表
SigPending、mask、备用信号栈	每线程独占	投递、sigprocmask、sigreturn 和 exec 更新	每个线程可独立屏蔽、等待和处理信号
SigHandler	同一线程组共享	sigaction 更新，exec 重置用户 handler	任一线程修改 handler 后全进程可见
TASK_MANAGER	全局 tid -> Weak<TCB>	创建时加入，逻辑退出时移除	kill/tkill/tgkill、调度和清理路径可按 tid 定位任务
children	线程组共享，保存子进程 leader 的 Arc<TCB>	子进程创建时加入，wait4 成功后删除	zombie 状态在父进程读取前不会丢失

图 2-1 展示了这些对象之间的关系。一个“进程”由线程组身份和一组共享资源共同构成；TCB 才是真正进入调度器的执行单元。

图 2-1 进程、线程与资源对象关系

```

进程 (tgid)
├─ ThreadGroup —Weak—> TCB(tid = tgid, leader)
│   │   └─ TCB(tid = n1)
│   │   └─ TCB(tid = n2)
├─ Arc<MemorySet> <— 同组线程通常共享
├─ Arc<FdTable> <— 同组线程共享；fork 默认复制表
├─ Arc<SigHandler> <— sigaction 进程级可见
└─ children: Arc<TCB> —> 保留子进程 wait status

```

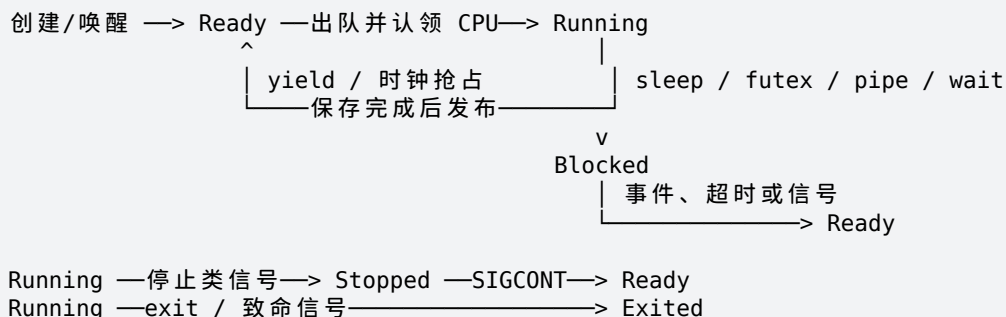
每个 TCB 独占：KernelStack + TrapContext + TaskContext + SigPending + SignalStack

2.3 任务状态与生命周期

2.3.1 状态机与调度不变量

RespOS 使用 Ready、Running、Blocked、Stopped 和 Exited 五种任务状态。图 2-2 中最重要的不变量是：同一个 tid 不能同时出现在 ready queue 与 blocked_tasks 中；多核取任务时，出队与 CPU owner 认领必须处于同一调度临界过程，避免其他 CPU 或唤醒者看到“已出队但仍可再次发布”的中间状态。

图 2-2 任务状态转换



队列锁只能保证容器本身不被并发破坏，不能自动保证状态机正确。RespOS 因此还通过 `task_index` 阻止重复入队，通过 `blocked_tasks` 保存真正睡眠的任务，并在 debug 构建中检查队列、索引和状态三者是否一致。

2.3.2 创建：fork、clone 与共享边界

`fork`、线程创建和 `vfork` 最终都收敛到 `sys_clone()`。系统调用层校验 `flag`，设置父/子 `tid`、`TLS`、用户栈和子任务返回值；`TaskControlBlock::clone_()` 构造身份及资源关系；最后由调度器发布新任务。表 2-3 只列出会改变对象身份或共享关系的关键 `flag`，常规 ABI 标志不在此展开。

表 2-3 clone 关键标志与当前共享语义

标志	未设置	设置后	当前实现边界
<code>CLONE_THREAD</code>	新 <code>tid</code> 同时成为新 <code>tgid</code> ，创建新线程组	新 <code>tid</code> 加入调用者线程组并沿用 <code>tgid</code>	要求 <code>CLONE_SIGHAND</code> ，后者又要求 <code>CLONE_VM</code>
<code>CLONE_VM</code>	通过 COW 建立子 <code>MemorySet</code>	共享同一个 <code>MemorySet</code>	非线程 <code>vfork</code> 为保护父映像，当前仍建立 COW 子空间
<code>CLONE_FILES</code>	新进程复制 <code>descriptor</code> 表	共享同一个 <code>FdTable</code>	同组线程无条件共享 <code>fd table</code>
<code>CLONE_VFORK</code>	父任务可在 <code>child</code> 发布后继续	父任务阻塞到 <code>child</code> 成功 <code>exec</code> 或 <code>exit</code>	已实现同步边；未直接共享父地址空间

代码片段 2-2 是资源选择的核心。这里的“复制 `fd table`”并不复制每个 `open-file description`：新表中的 `FdEntry` 仍持有相同的 `Arc<FileOp>`，因此 `fork` 后父子可以独立 `close` 或设置 `CLOEXEC`，但共享文件偏移和管道端点；`CLONE_FILES` 则连 `descriptor` 表本身也共享。

代码片段 2-2 clone 的地址空间与 `fd table` 选择

```

// os/src/task/task.rs
let memory_set = if flags.share_user_vm() {
    self.memory_set.clone()
} else {
    Arc::new(RwLock::new(MemorySet::from_existed_user(
        &mut self.memory_set.write(),
    )))
};

let fd_table = if is_thread || flags.contains(CloneFlags::CLONE_FILES) {
    self.fd_table.lock().clone()
} else {
    FdTable::from_existed_user(&self.fd_table.lock())
};
  
```


普通 fork 的内存复制采用 COW。MemorySet::from_existed_user() 复制 VMA 元数据；未驻留的 lazy 页继续保持未分配，只读页和共享页复用原 frame，私有可写驻留页在父子两侧转为 COW。实现先完成子页表构造，再修改父页表权限，使中途 ENOMEM 不会在父地址空间留下映射空洞。这一机制把 fork 成本从“复制整个地址空间”降为“复制元数据并按首次写入付费”。

vfork 的难点不只是“让父任务睡眠”，而是建立不会丢失的一次性同步边。若先发布 child 再登记 parent，child 可能在另一 CPU 上立即 exec 并发送唤醒，而 parent 尚未进入 blocked 集合。RespOS 按图 2-3 固定发布顺序，并用 vfork_parent: Option<Weak<TCB>> 保证 exec/exit 只释放一次。

图 2-3 vfork 的一次性同步顺序

```
parent: new_task.set_vfork_parent(parent)
        -> prepare_current_task_blocked(parent)
        -> add_task(child)
        -> switch_to_next_task()

child : exec 完整提交 或 exit
        -> release_vfork_parent()
        -> wakeup_task(parent.tid)
```

Linux vfork 还要求 child 在 exec/exit 前直接使用父地址空间。RespOS 当前将地址空间保存在 Arc<RwLock<MemorySet>> 中，而 exec 通过替换锁内的 MemorySet 安装新映像；若非线程 vfork 直接共享该对象，child exec 会同时替换父任务可见的映像。因此当前实现保留了父子同步语义，但用 COW 子空间隔离映像替换。这是明确的数据结构边界，而不是将当前行为误写成完整 Linux vfork。

2.3.3 exec：先准备、再静止、后提交

exec 的目标是一次性替换整个进程映像，而不是在旧地址空间上逐段修补。RespOS 将文件 ELF 加载收敛到 MemorySet::try_from_elf_file()，将安装阶段收敛到 install_exec_image()，按表 2-4 的四个阶段推进。

表 2-4 exec 映像替换阶段与正确性保证

阶段	主要工作	正确性保证
准备新映像	校验 ELF, 创建新 MemorySet, 布置 argv/envp/auxv	失败发生在提交前，旧程序仍保持完整
静止 sibling	设置 terminate_requested, 从调度器摘除并等待 cpu_owner 释放	其他 CPU 不再执行旧用户映像
提交映像	替换 MemorySet、激活页表、重建 trap context	PC、SP、寄存器与新地址空间同时生效
提交外围状态	私有化 fd table、执行 CLOEXEC、重置信号状态、释放 vfork parent	父任务只会在 exec 可见状态完整后恢复

图 2-4 展示了多线程 exec 中的 stop/ack 协议。sibling 的 robust list 和 clear-child-tid 保存的是旧映像用户地址，因此必须在地址空间替换前处理；仅从 ready queue 删除任务也不够，因为它可能仍在远端 CPU 上执行。远端 CPU 只有在 __switch 保存完上下文并回到 per-CPU idle 栈后才释放 cpu_owner，publish_saved_handoff() 对已经请求终止的任务不再重新发布。

图 2-4 多线程 exec 的静止与提交顺序

```

准备完整新映像
└> request_termination(siblings)
    └> remove_task(siblings)
        └> 等待所有 cpu_owner 释放
            └> 在旧 MM 中清理 robust list / clear-child-tid
                └> 替换 MemorySet 与 TrapContext
                    └> CLOEXEC + signal reset + 唤醒 vfork parent

```

文件式 exec 还采用 private file-backed VMA 按需加载主程序的 PT_LOAD。内核只读取 ELF header、program header 和 PT_INTERP 元数据，元数据前缀上限为 1 MiB；程序页面在首次访问时按页读入。与早期 read_all() 整文件载入相比，这一设计避免约 45 MiB 的 cargo 可执行文件在内核堆与用户页中重复占用。argv 和 envp 每组允许最多 4096 项，同时以每组 1 MiB 累计字节限制约束内核分配，既支持真实工具链，又避免无界输入。

当前 exec 只允许线程组 leader 执行，以避免非 leader 替换映像后重新组织 tgid 与父子关系；动态链接器仍采用整文件读取，尚未复用主程序的按需路径。这两个边界不影响上述提交顺序，但限定了当前兼容范围。

2.3.4 exit、wait 与延迟析构

RespOS 将“停止执行”“向父进程保留退出状态”和“释放 TCB/内核栈”分成三个不同时间点。表 2-5 说明了每个时间点的资源处理职责；这种分离是 wait 语义和内核栈安全共同要求的结果。

表 2-5 退出与回收的分阶段职责

时间点	状态变化	主要处理
线程逻辑退出	当前 TCB 变为 Exited	清理 robust list、clear-child-tid、futex waiter，并从调度器/线程组/任务索引摘除
线程组退出	原子选出唯一 teardown owner	静止 sibling，处理共享 MM/fd/timer，托管孤儿，leader 向父进程发布 SIGCHLD 与 wait status
wait4 成功	父任务删除 children[child_tid]	先 copyout status/rusage，成功后再删除 zombie 并累计子进程时间
安全栈上析构	idle loop 清空 DEAD_TASKS	最终 drop TCB，回收内核栈 slot；已有内核页表映射保留供复用

exit_group() 和致命信号共用组级退出路径。共享的 group_exiting 通过 compare-exchange 选出唯一清理者，防止多个 CPU 重复清空 fd table、回收地址空间或通知父进程。组级退出沿用 exec 的 stop/ack 协议：先阻止 sibling 再次被 claim，再等待远端 owner 释放，最后处理共享资源。

wait4 采用 prepare/copyout/commit。内核先将 status 和 rusage 写回用户空间，所有 copyout 成功后才从 children 删除 child 并提交子进程时间；用户指针无效时，父任务可修正参数后再次 wait，不会丢失退出状态。SMP 下实际 waiter 可能是父线程组中的任意线程，因此子进程退出会唤醒所有设置了 waiting_for_child 的成员；waiter 在发布 blocked 后还会复查 exited_children 和中断标志，覆盖“扫描完成、正式睡眠之前 child 恰好退出”的窗口。

图 2-5 展示了父子回收与本任务析构两条并行生命周期。父进程回收的是用户可见的 zombie 状态，idle loop 回收的是已经不再执行的内核对象，两者不能互相替代。

图 2-5 退出通知、wait 与最终析构

```

child 逻辑退出 → Exited + SIGCHLD → parent wait4 copyout → children 删除
|
└─> 当前 CPU 切到 idle 栈 → DEAD_TASKS → cleanup_dead_tasks → drop TCB/栈 slot

```

2.4 分层优先级调度

RespOS 的调度器没有照搬 Linux CFS，而是采用便于检查的分层固定优先级队列，在同一框架下承载实时任务、普通任务和低优先级后台任务。表 2-6 描述了当前选择顺序和兼容边界。

表 2-6 调度策略与就绪队列

任务类型	队列组织	选择顺序	当前语义边界
SCHED_FIFO / SCHED_RR	优先级 1~99 的 RT 队列	数值越大越优先，同级 FIFO	周期时钟当前会重新排队两者，尚未完整区分 Linux FIFO 与 RR
SCHED_OTHER / SCHED_BATCH	nice -20~19 的 40 个普通队列	nice 越小越优先，同级轮转	固定优先级模型，不维护 CFS 虚拟运行时间
SCHED_IDLE	单独的 idle 队列	RT 与普通队列均无可运行任务时选择	用于最低优先级后台工作

代码片段 2-3 展示了调度器的核心容器。RT 和普通队列各有位图，选择任务时可以跳过空优先级；`task_index` 记录 `tid` 当前所在的 ready queue，既支持快速删除，也用于拒绝重复入队；真正睡眠的任务保存在独立的 `blocked_tasks` 中。

代码片段 2-3 分层调度器的数据结构

```

// os/src/task/scheduler.rs
pub struct Scheduler {
    rt_queues: Vec<VecDeque<Arc<TaskControlBlock>>>,
    normal_queues: Vec<VecDeque<Arc<TaskControlBlock>>>,
    idle_queue: VecDeque<Arc<TaskControlBlock>>,
    rt_bitmap: u128,
    normal_bitmap: u64,
    task_index: HashMap<usize, ReadyQueue>,
    blocked_tasks: HashMap<usize, Arc<TaskControlBlock>>,
}

```

时钟抢占和主动 `yield` 都会经过 per-CPU handoff，再由 idle loop 将仍为 **Running** 且未被请求终止的任务改为 **Ready** 并放回队尾。因此同优先级内形成简单 round-robin，不同优先级之间仍保持 RT → normal → idle 的严格选择顺序。

2.5 上下文切换与内核栈安全

RespOS 采用同步有栈的内核任务切换。用户态 trap 保存完整的 `TrapContext`；真正的内核任务切换以函数调用为边界，只需保存调用约定中的 callee-saved 寄存器、线程指针和页表 token。代码片段 2-4 给出了跨架构共用的 `TaskContext` 表示：RISC-V 的 `s0-s11` 与 LoongArch 的对应保存寄存器统一落在 `s[12]` 中，架构汇编负责具体映射。

代码片段 2-4 内核任务上下文

```
// os/src/task/context.rs
#[repr(C, align(16))]
pub struct TaskContext {
    ra: usize,
    tp: usize,
    s: [usize; 12],
    mmu_token: usize,
    _padding: usize,
}
```

内核栈按 slot 分配，相邻 slot 之间保留一个 guard page。任务析构时回收 slot，但保留全局内核页表中的栈映射；后续任务复用同一 slot 时无需频繁修改共享内核页表。这一选择把短进程压力下的页表扰动转化为有限 slot 的循环复用。

多核切换的关键约束是：正在旧 CPU 上执行的任务不能直接加入全局 ready queue。早期“先入队、再 __switch”的顺序允许另一 CPU 立即取到该任务，恢复一份尚未保存完成的 context。RespOS 通过 per-CPU handoff 和 cpu_owner 将顺序固定为图 2-6 所示。

图 2-6 context-save 到 ready-publication 的 handoff 协议

旧 CPU / task 栈	旧 CPU / idle 栈	新 CPU
Running task		
↳ 移入本 CPU handoff		
↳ __switch 保存 context		
	↳ publish_saved_handoff	
	↳ 若仍可运行：set_ready + enqueue	
	↳ release cpu_owner	
		↳ claim owner
		↳ __switch 恢复

handoff 保证 ready 发布发生在上下文保存之后，cpu_owner 则保证新 CPU 的 claim 发生在旧 CPU 明确释放之后。即使 wakeup 在保存期间提前到达，任务也只会留在 ready queue 中等待 owner 释放，不会被恢复两次。

2.6 多核处理器管理

RV64 的每颗 CPU 都有独立的 Processor、idle task 和 handoff slot；LoongArch 当前仍以单核处理器数组入口运行。没有可运行任务时，RV64 CPU 先发布 idle 状态，再检查一次全局 ready queue，仍为空才打开本地 timer 中断并进入 WFI。第二次检查覆盖了“首次取队列失败”到“CPU 正式睡眠”之间的新任务入队窗口。

图 2-7 给出了跨核唤醒链路。IPI 只负责唤醒 WFI 中的 CPU，任务选择仍由 idle loop 完成，从而避免在中断上下文中执行复杂调度逻辑。

图 2-7 多核任务唤醒与定向 IPI

```
事件到达
↳ blocked_tasks 删除 tid
↳ task.set_ready + 加入原优先级队列
↳ 按 CPU affinity 选择在线 idle hart
↳ IPI 唤醒
↳ idle loop fetch_for_cpu + claim cpu_owner
```

affinity 必须同时约束“谁能取任务”和“唤醒谁”。`fetch_for_cpu()` 会在优先级队列内跳过当前 CPU 不允许或 owner 尚未释放的任务，同时保留其相对位置；任务入队和 owner 释放后的 kick 也只面向 affinity 允许的在线 idle CPU。后一处补发 kick 覆盖了第一次 IPI 到达过早、任务仍被旧 owner 持有的窗口。

当前调度器使用全局锁和全局 run queue。这使 ready/blocked/index 不变量集中且容易审查，也支持当前 RV64 多核正确性；当 CPU 数继续增长时，全局锁争用会成为引入 per-CPU run queue 与负载均衡的主要依据。

2.7 跨模块协作与功能总结

进程管理并不独立完成所有任务语义，而是为内存、文件、信号、时钟和 IPC 提供统一生命周期边界。表 2-7 总结了这些协作关系及本章所证明的设计成果。

表 2-7 进程管理的跨模块接口与设计成果

协作模块	进程管理提供或依赖的接口	已形成的能力
内存管理	MemorySet 的 COW clone、exec 替换、active-hart 标记与回收	fork 延迟复制、文件 ELF 按需加载、SMP 地址空间安全切换
文件系统	FdTable 复制/共享、CLOEXEC、退出清表	fork 共享 open-file 状态，exec 不误关共享父表，pipe EOF 可按最后引用产生
信号	每线程 pending/mask、组共享 handler、可中断等待	kill/tkill/tgkill、停止/继续/致命信号与 wait 唤醒接入统一 TCB
futex/pipe/wait	blocked 注册、single-winner 完成、signal interrupt	事件、超时、signal 和退出竞争时不重复唤醒或永久睡眠
时钟与 SMP	抢占入口、per-CPU idle、affinity IPI、cpu_owner	分层调度、跨核唤醒和 context handoff

综上，RespOS 的进程管理不是一组孤立系统调用，而是一套围绕 TCB 生命周期建立的执行模型：tid/tgid 定义身份，Arc/Weak 定义所有权，scheduler 定义状态发布，handoff/owner 定义多核切换边界，wait 与 DEAD_TASKS 分别完成用户可见回收和内核对象析构。正是这些边界共同保证了 fork/exec/exit、信号和 IPC 能在同一任务模型上稳定协作。

3. 中断与异常处理

本章回答：用户态或内核态发生 trap 后，RespOS 如何保存现场、识别原因、调用相应内核服务，并在保证现场完整的前提下返回？

3.1 模块定位

中断与异常处理是用户程序进入内核的入口，也是时钟、系统调用、缺页和进程抢占等机制的连接点。RespOS 将中断与异常处理过程分为两层：架构相关的入口与返回代码负责完成寄存器保存、特权级切换和硬件状态恢复；架构无关的 Rust 代码负责识别异常原因，并调用系统调用、内存管理、信号和调度模块。

这种划分使得两种架构可以共享同一套处理策略，同时保留各自必要的硬件细节。图 3-1 展示了一次用户态 trap 的完整路径。

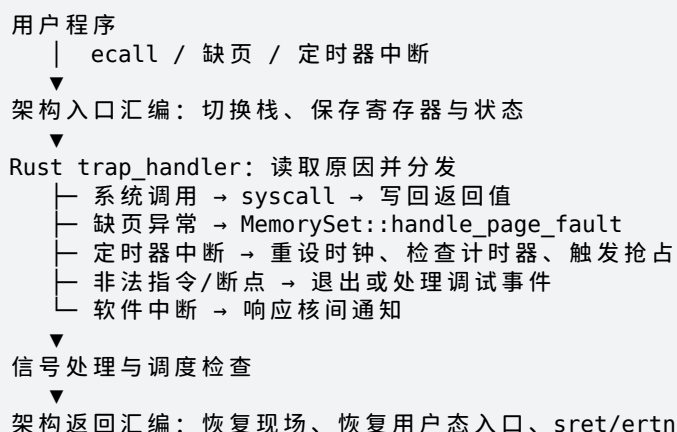


图 3-1 用户态 trap 的入口、分发与返回路径

3.2 Trap 的基本流程

一次 trap 可以概括为“入口保存、Rust 分发、后置检查、返回恢复”四个阶段。系统调用、缺页、非法指令和断点属于同步异常，定时器和软件中断则可能在程序执行过程中异步到达；无论来源如何，最终都进入同一条处理链路。

首先，硬件将返回地址和异常原因写入架构寄存器，并跳转到预先配置的 trap 入口。入口汇编切换到内核栈，保存通用寄存器和返回所需的状态，构造 TrapContext，再调用 Rust 侧的 trap_handler。RISC-V 和 LoongArch 使用的寄存器与入口指令不同，但这一阶段都只负责建立一份完整、可恢复的内核现场。

随后，trap_handler 读取硬件原因并完成分发：系统调用交给 syscall 层，缺页交给 MemorySet，定时器负责设置下一次触发并推进超时处理，软件中断负责响应核间通知。处理结果可能是修改 TrapContext 后继续执行，也可能是向用户任务递送信号、结束任务，或在安全点进入调度器。用户态异常通常通过信号或退出隔离应用故障；内核态发生非法指令、缺页或系统调用则按内核错误处理。

在返回前，内核统一完成待处理信号和必要的调度检查。若当前任务仍可运行，返回汇编就按照固定布局恢复 `TrapContext`，切回用户态 `trap` 入口，并执行 RISC-V 的 `sret` 或 LoongArch 的 `ertn`；若任务已经阻塞、退出或被抢占，则先由调度器切换到其他任务，再从新任务的现场恢复。系统调用返回值、缺页修复结果以及信号处理后的用户态控制流，最终都通过这份可修改的现场生效。

3.3 `TrapContext`：统一的可恢复现场

`TrapContext` 描述了用户程序的运行状态，定义了入口汇编与 Rust 处理逻辑之间的数据契约。入口汇编按照固定顺序把寄存器保存到内核栈，Rust 代码通过这个结构读取系统调用参数、修改返回值或调整返回地址，最后由返回汇编按照相同布局恢复。

3.3.1 RISC-V64 的 `TrapContext`

RISC-V64 的上下文需要保存通用寄存器、浮点寄存器、浮点控制状态以及用户态返回所需的 `sstatus` 和 `sepc`：

```
#[repr(C)]
#[derive(Debug, Clone, Copy)]
pub struct TrapContext {
    pub x: [usize; 32],
    pub sstatus: Sstatus,
    pub sepc: usize,
    pub f: [usize; 32],
    pub fcsr: usize,
    _padding: usize,
}
```

代码片段 3-1 RISC-V64 的 `TrapContext` 结构

其中，`x[10]` 是 RISC-V ABI 的 `a0`，既用于传递第一个参数，也用于接收系统调用返回值；`x[2]` 是用户栈指针 `sp`；`sepc` 保存 `trap` 返回地址，`sstatus` 保存返回时需要恢复的特权级和中断状态。浮点寄存器也属于用户程序的执行现场，因此入口和返回路径需要一并保存、恢复。

创建用户初始上下文时，RespOS 将 `sstatus.SPP` 设置为 `User`，并关闭保存现场期间的 `SIE`；同时设置入口地址和用户栈指针。对于 Linux ABI 程序，`a0` 等参数由用户栈上的启动数据解释；内置教学程序则可以使用寄存器传递 `argc`、`argv` 等参数。

3.3.2 LoongArch64 的 `TrapContext`

LoongArch64 的特权状态和返回地址分别由 `prmd` 与 `era` 表示，当前上下文结构保存通用寄存器以及这两个架构状态寄存器：

```
#[repr(C)]
#[derive(Debug, Clone, Copy)]
pub struct TrapContext {
    pub x: [usize; 32],
    pub prmd: usize,
    pub era: usize,
}
```

代码片段 3-2 LoongArch64 的 `TrapContext` 结构

LoongArch 的 `a0` 对应 `x[4]`，栈指针对应 `x[3]`，系统调用号位于 `x[11]`，六个参数位于 `x[4..10]`。`prmd` 中的 `PPLV` 和 `PIE` 位用于记录异常前的特权级及返回后的中断状态。

创建 LoongArch 用户上下文时，RespOS 将 `prmd` 初始化为用户态并允许异常返回后重新开启中断，再设置 `era` 和用户栈指针。与 RISC-V 一样，Linux ABI 程序的启动参数从用户栈获取；内置程序则由寄存器传入启动参数。

3.3.3 两种上下文共同语义

两种结构在字段层面并不完全相同，但它们都提供了访问返回值、用户栈和返回 PC 的语义接口。

图 3-2 总结了 `TrapContext` 在一次 `trap` 中的生命周期。

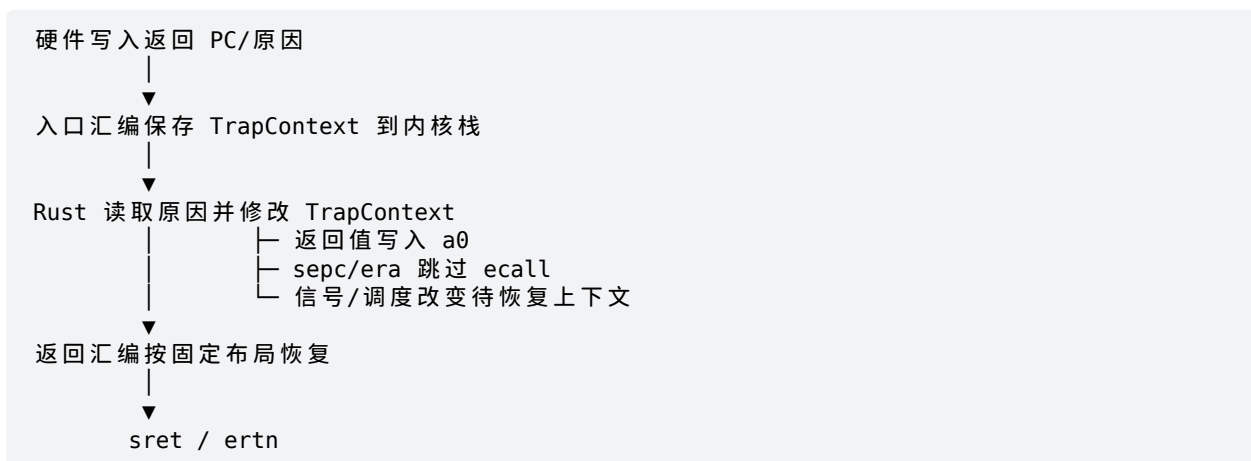


图 3-2 `TrapContext` 在一次 `trap` 中的生命周期

3.4 中断屏蔽与并发安全

`trap` 处理不仅要识别事件，还要避免处理过程被同类事件破坏。RespOS 为两种架构提供统一语义的 `InterruptGuard`：构造时保存当前中断使能状态并关闭中断，离开作用域时仅在原先开启的情况下恢复。这样既能保护临界区，也不会无条件打开调用者原本关闭的中断。

场景	RISC-V 机制	LoongArch 机制	共同语义
trap 入口	配置 <code>stvec</code> ，进入内核入口	配置 <code>EENTRY</code> ，进入内核入口	先建立内核可用的保存环境
中断总开关	<code>sstatus.SIE</code>	<code>CRMD.IE</code>	由 <code>guard</code> 保存并按原状态恢复
用户态返回	<code>sret</code> 根据 <code>sstatus</code> 返回	<code>ertn</code> 根据 <code>PRMD/ERA</code> 返回	只在现场完整后恢复用户态
定时器状态	SBI 设置下一次触发	清除定时器状态并配置下一次触发	重新武装后再执行计时器服务

表 3-2 双架构 `trap` 控制语义对照

RISC-V 在返回路径强调顺序性约束：恢复寄存器的中间阶段优先保持 `SIE` 关闭，待通用寄存器和浮点寄存器全部恢复后，才切换回用户态。否则定时器可能在现场尚未恢复完成时再次进入 `trap`，导致使用半恢复状态的寄存器。LoongArch 则在 `ertn` 前准备好 `PRMD`、`ERA`、用户态入口和栈切换信息，再由硬件完成特权级返回。

设备外部中断的具体控制由架构和驱动层共同承担：架构层负责把硬件中断送入统一 `trap` 路径，驱动层负责识别设备、清除设备状态并完成请求。这样可以把“如何进入内核”和“哪个设备需要服务”分开，避免 `trap` 分发器依赖具体设备实现。

3.5 功能与设计总结

设计点	实现方式	对系统的作用
统一入口	架构汇编保存现场, Rust 统一分发	系统调用、异常和中断共享清晰的边界
可修改现场	<code>TrapContext</code> 作为返回上下文	返回值、信号、抢占都可通过同一机制生效
用户/内核分流	用户错误转信号或退出, 内核错误 <code>panic</code>	隔离应用故障, 尽早暴露内核不变量破坏
架构复用	共享处理策略, 保留各自 CSR 与返回指令	支持 RISC-V 与 LoongArch 的一致功能语义
返回安全	固定保存布局、屏蔽顺序和栈切换协议	避免重入、现场损坏和错误特权级返回

中断与异常模块承担了一条完整的控制流: 将硬件事件转换为内核服务, 把服务结果写回可恢复现场, 最后将控制权安全地交还给任务。内存、时钟、进程、信号和驱动模块则分别在这条闭环的分发节点上实现各自功能。

4. 内存管理

本章回答：物理内存如何分配？进程地址空间如何表示？缺页、COW 与文件映射如何在同一套模型中协作？多核下又如何保证页表修改可见？

RespOS 采用“MemorySet 拥有地址空间语义”的设计，因为单独管理页表只能说明 CPU 当前看见什么，无法表达一段虚拟地址是否合法、是否按需分配、来自哪个文件，以及由谁回收。因此，内存层将 VMA (Virtual Memory Area)、PTE 和物理页帧组成一个完整的所有权模型，并让 `mmap`、`fork`、`exec`、用户指针访问与进程退出共用这套规则。

4.1 分层对象与核心不变量

RespOS 将“虚拟地址承诺”、“硬件映射状态”和“物理内存所有权”分层表示。如图 4-1 所示，一个已声明但尚未访问的 lazy VMA 可以没有 PTE 和页帧；只有页面真正被访问后，后两层状态才建立。

```
MemorySet (一个地址空间)
├─ MapArea / VMA: 地址范围、权限、shared/private、file backing
│   └─ data_frames: 该 VMA 中已驻留页面的所有权
├─ PageTable / PTE: CPU 当前可见的 VPN → PPN 和硬件权限
└─ active_hart_mask (RV64): 当前正在使用该页表的 hart
```

图 4-1 MemorySet、VMA、PTE 与驻留页的层次关系

表 4-1 进一步给出四类对象的职责和生命周期。其中，VMA 是内核判定缺页能否被修复的依据，而不是仅用来加速查找的缓存。

表 4-1 地址空间核心对象

对象	表达的概念	拥有的状态	创建、共享与释放
MemorySet	一个进程地址空间	页表、VMA 集合、brk、mmap_start	exec 构建，CLONE_VM 任务共享，最后一个引用释放
MapArea	一段连续 VMA	VPN 范围、权限、共享标志、文件偏移、驻留页	ELF loader、brk、mmap 创建；munmap/mprotect/mremap 切分
PTE	硬件页级映射	PPN、R/W/X/U 和 COW 等标志	建图或缺页时创建，改权限时更新，解除映射时删除
FrameTracker	一个 4 KiB 物理页帧的所有权	PPN；共享页由 Arc 表达引用	分配时清零，最后一个所有者销毁时自动归还

代码片段 4-1 地址空间与 VMA 的关键字段

```
// os/src/mm/memory_set.rs (节选)
pub struct MemorySet {
    pub brk: usize,
```

```

pub heap_bottom: usize,
pub mmap_start: usize,
pub page_table: PageTable,
areas: Vec<MapArea>,
#[cfg(target_arch = "riscv64")]
active_hart_mask: AtomicUsize,
}

struct MapArea {
    vpn_range: VPNRange,
    data_frames: BTreeMap<VirtPageNum, Arc<FrameTracker>>,
    map_type: MapType,
    map_perm: MapPermission,
    shared: bool,
    locked: bool,
    file_backing: Option<FileBacking>,
    shm_attach_id: Option<usize>,
    // 另有 wipe_on_fork / dontfork 等 VMA 语义标志
}

```

代码片段 4-1 中的 `areas` 是按起始 VPN 排序的 VMA 序列，`data_frames` 则是稀疏的驻留页集合。这种“范围元数据 + 实际工作集”的表示使大范围 lazy VMA 不需要按虚拟跨度预先创建管理对象，退出回收也只遍历已驻留页。

RespOS 在 debug 构建中对上述模型执行结构自检：VMA 必须有序、非空且不重叠；`data_frames` 的 VPN 必须落在所属 VMA 中并对应有效 PTE；用户 VMA 必须带 U 权限；COW 页不能同时可写；shared VMA 不能误入私有 COW 路径。这些断言把“实现约定”变成了可执行的不变量。

4.2 双架构地址空间布局

RespOS 在 RV64 和 LA64 上保持共享的 39 bit 虚拟地址模型，因为这能让 `MemorySet`、VMA 和大部分内核映射逻辑在两个架构上共用。两种架构的内核都链接到 `0xffff_ffc0_0000_0000` 起始的高半区，用户映射位于低半区，页大小均为 4 KiB；具体差异见表 4-2。

表 4-2 RV64 与 LA64 内存布局对比

项目	RISC-V 64	LoongArch 64
内核虚拟基址	<code>0xffff_ffc0_0000_0000</code>	<code>0xffff_ffc0_0000_0000</code>
内核物理入口	<code>0x8020_0000</code>	<code>0x0020_0000</code>
用户 mmap 区间	<code>0x20_0000_0000 ~ 0x22_0000_0000</code>	相同
用户信号返回跳板	<code>0x3f_ffff_e000</code>	相同
内核堆	64 MiB	48 MiB
物理内存上限来源	OpenSBI 传入 FDT 的 <code>/memory/reg</code>	板级静态配置

内核页表对 `text`、`rodata`、`data`、`bss` 分别设置 RX、R、RW、RW 权限，其余 RAM 和 MMIO 按 `KERNEL_BASE + physical_address` 的固定偏移映射。因此，内核可以由 PPN 直接得到访问页帧的高半区虚拟地址，页表页构造、COW 复制和用户拷贝无需临时建立 `kmap`。

LoongArch 启动时确实会使用早期低地址可达路径，但进入公共内存初始化后会激活高半区内核页表并关闭低地址 direct map。因而，当前实现不是“RV64 使用高半区页表、LA64 长期使用 DMW 内核”的两套公共模型；架构差异主要保留在页表格式、MMU 寄存器与 TLB 操作后端中。

4.3 物理页帧与内核堆

RespOS 将物理页帧分配和内核小对象分配彻底分开。页表页、用户驻留页和内核映射页最终需要 4 KiB 物理帧；Vec、Arc、BTreeMap 等 Rust 对象则从 buddy system 管理的固定内核堆中获得可变大小空间。

代码片段 4-2 **FrameTracker** 的 RAII 回收

```
// os/src/mm/frame_allocator.rs (节选)
pub struct FrameTracker {
    ppn: PhysPageNum,
}

impl FrameTracker {
    pub fn new(ppn: PhysPageNum) -> Self {
        let mut frame = Self { ppn };
        frame.clear();
        frame
    }
}

impl Drop for FrameTracker {
    fn drop(&mut self) {
        FRAME_ALLOCATOR.lock().dealloc(self.ppn());
    }
}
```

代码片段 4-2 体现了页帧所有权的两个安全性质：新页必先清零，避免前一个使用者的数据泄漏；页帧随最后一个 **FrameTracker** 所有者销毁而归还，COW 和共享映射只需通过 `Arc<FrameTracker>` 表达共享，无需另外维护一套手工页帧引用计数。底层页帧分配器是 **StackFrameAllocator**：优先复用回收栈中的 PPN，再从未分配区间线性取页；它不是 buddy allocator。

内核堆使用 `buddy_system_allocator::LockedHeap`，但在外层增加了 `IrqSafeHeap`。这一包装不仅使用自旋锁串行化不同 CPU，还在每次分配、释放以及直接获取堆 guard 时关闭本地中断。之所以需要第二层保护，是因为跨 CPU 自旋锁不能阻止同一 CPU 在持锁期间被 timer interrupt 打断并再次申请堆；恢复中断前先释放堆锁，从而避免同核重入自锁。

4.3.1 RV64 动态物理内存发现

RISC-V 的物理内存管理采用“最大可达窗口”与“实际可分配范围”分离的设计。早期根页表预留最多 8 GiB QEMU RAM 的可达窗口，启动 hart 再从 OpenSBI 传入的 FDT `/memory/reg` 取得本次启动的真实内存末址。页帧分配器只管理真实范围，因此既能使用 8 GiB 负载，也不会在 256 MiB 配置下误分配不存在的物理页。

```
early page table: 建立最大 8 GiB 可达窗口
    ↓
FDT `/memory/reg`: 发布本次启动的真实 RAM 末址
    ↓
frame allocator: 只管理 [kernel_end, physical_memory_end)
    ↓
kernel direct map 与 `/proc/meminfo`: 使用同一真实上限
```

图 4-2 RV64 可达范围与可分配范围的分离

图 4-2 中的两级边界还避免了 8 GiB 最大窗口被误当成可分配内存。为避免 direct map 本身消耗大量页表页，首个 RAM GiB 保留 4 KiB 映射，以继承内核各 section 的细粒度权限；后

续完整 GiB 使用 Sv39 level-2 的 1 GiB 叶页。这一优化只影响页表成本，不改变上层的页帧和 VMA 模型。

4.4 VMA 变换与映射类型

RespOS 把 `mmap`、`munmap`、`mprotect`、`mremap`、`MAP_FIXED` 对地址空间的修改收敛在 `MemorySet` 中，因为这些操作共享同一组范围校验、VMA 切分和页帧转移规则。`syscall` 层只解析 Linux ABI 参数、用户指针、`fd` 和 `errno`，不再维护第二套地址空间状态机。

当操作只覆盖某个 VMA 的中间区间时，`split_by_overlap` 统一将原 VMA 分成左、中、右三段。切分不仅改虚拟范围，还同步分配 `data_frames`、修正 `file backing` 偏移和有效长度，并保留 `shared`、`locked`、`wipe_on_fork`、`dontfork` 等元数据。`debug` 启动时会运行头部、尾部、中段和二次切分自测，防止系统调用各自实现切分时出现语义漂移。

不同 VMA 的建图与派生策略汇总于表 4-3。

表 4-3 主要 VMA 类型及其建图策略

VMA 类型	建立时	首次访问	fork 后关系
私有匿名 <code>mmap</code>	只记录 VMA	分配清零页	已驻留可写页进入 COW，未驻留页各自缺页
<code>brk</code> 堆	RV64 记录 lazy VMA；LA64 当前预先驻留	按架构策略处理	私有可写页进入 COW
私有文件映射	记录 <code>file/offset/len</code>	分配私有页并按页读文件	已驻留可写页进入 COW，修改不回写原文件
共享匿名映射	预先建立共享页	直接访问共享 frame	父子保持共享，不进入 COW
共享文件映射	在 MM 锁外预取并按文件页身份复用 frame	直接访问共享 frame	父子保持共享，解除映射前进入写回协议
文件系统主 ELF 段	<code>exec</code> 只记录 <code>PT_LOAD</code> backing	按页读取文件，BSS 剩余部分保持为零	<code>exec</code> 前的派生遵循私有文件页规则

需要特别注意的是，当前 ELF loader 为用户栈创建的是已驻留映射，并在栈底下方保留守护页；因此，本章不将“用户栈懒分配”列为当前特性。不同映射类型的差异由 `backing` 和 `shared` 语义决定，而不是根据调用这些对象的系统调用路径临时推测。

4.5 缺页处理与按需分配

RespOS 将合法缺页视为延迟工作的统一入口。RV64 和 LA64 的异常编码不同，但架构 `trap` 层都将其归一为取指、读取和写入三类 `PageFaultCause`；公共内存层再由故障地址定位 VMA，并根据访问类型检查 R/W/X/U 权限。

用户态 page fault

```

↓
地址位于用户上界内，且存在匹配 VMA？
├ 否：返回 EFAULT，交由 trap/signal 路径处理
└ 是：VMA 权限允许本次取指/读/写？
    ├── 否：拒绝访问
    └ 是
        ├── PTE 有效 + COW 写：恢复写权限或复制新页
        └ 是：继续访问
  
```

- └ PTE 缺失：分配匿名页或按页读取 file backing
- └ PTE 已被其他 CPU 补好：本地 fence 后重试指令

图 4-3 `handle_page_fault` 决策流程

图 4-3 的最后一个分支是多核环境下的必要补充。页错误可能已被当前 CPU 锁存，但当它等待 `MemorySet` 写锁时，另一个 CPU 可能已完成映射。内存层会重新检查最终 VMA 和 PTE；如果权限已满足，则只刷新本地 TLB 并重试原指令，避免把一次已被其他 CPU 修复的缺页误判为越界。

用户指针访问复用同一套缺页语义。`copy_from_user` 和 `copy_to_user` 先使用检查加法、页对齐和用户上界的范围函数验证整段缓冲区，再确保每个 lazy/COW 页可访问，最后通过 PTE 对应物理页逐页复制。内核不直接构造用户 slice，因此既能处理跨页和跨相邻 VMA 的缓冲区，也不会因直接解引用一个尚未驻留的用户地址而在内核态再次缺页。

4.6 写时复制

COW 将 `fork` 的成本从“复制全部私有页”改为“只复制后续真正被写的页”。之所以这样设计，是因为许多子进程会很快调用 `exec` 或退出，在 `fork` 时立即复制全部驻留页往往不会产生用户可见的价值。

`MemorySet::from_existed_user` 处理三种情况：已驻留的私有可写页由父子共享同一 `Arc<FrameTracker>`，两个 PTE 都去掉 `W` 位并设置软件 COW 标志；私有只读页直接共享且保持只读；尚未驻留的 lazy 页只复制 VMA 元数据，父子以后分别缺页。shared VMA 则始终共享原 frame，不进入私有 COW 路径。

代码片段 4-3 COW 缺页的单所有者与多所有者分流

```
// os/src/mm/memory_set.rs (等价节选)
let old_frame = area.data_frames.get(&vpn).ok_or(Errno::EFAULT)?;
if Arc::strong_count(old_frame) == 1 {
    page_table.modify_pte(vpn, PTEFlags::from(area_perm));
    page_table.clear_pte_cow(vpn);
} else {
    let old_data = old_frame.ppn().get_bytes_array();
    area.remap_one_with_data(page_table, vpn, old_data)?;
}
```

如代码片段 4-3 所示，写缺页时如果当前进程已是页帧的唯一所有者，只需恢复 `W` 位；仍有其他所有者时，才申请新页、复制 4 KiB 数据并替换当前 PTE。新页在替换前已分配且填充完成，因而 `ENOMEM` 不会在原地址空间中留下空洞。

`fork` 建立 COW 时也遵循页粒度的先准备后提交：先把子进程 PTE 映射到共享页帧，成功后再将该页的父 PTE 改为 COW。若后续页处理失败，已转换的父页可能保留为只读 COW，但它仍然指向原数据并可在下次写入时正常恢复；这保证了失败路径不丢页、不丢数据，而不夸大为全地址空间一次性事务。

4.7 共享文件映射与锁外写回

可写 `MAP_SHARED` 的难点不是建立 PTE，而是同一文件页的多个映射如何看见同一份修改，以及 `msync`、`munmap`、`MAP_FIXED`、`mremap`、`mprotect` 和进程退出如何共用同一条写回协议。RespOS 以

(dev, ino, page_index) 作为共享文件页身份，用弱引用缓存复用存活的 `FrameTracker`，使同一文件页的不同映射共享物理内容。

写回使用“锁内快照、锁外 I/O、成功后改图”的协议。如图 4-4 所示，内存层持 `MemorySet` 锁时只校验范围并把驻留页内容复制到独立 `FileWriteback` 快照；锁释放后才经 `FileOp` 写入页缓存，`MS_SYNC` 再额外执行 `fsync`；对能向用户返回错误的映射 `syscall`，写回成功后才解除或替换映射。

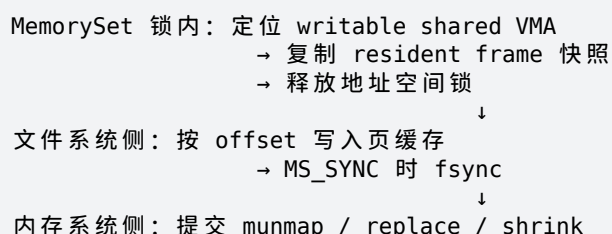


图 4-4 共享文件页的锁外写回协议

之所以不在 `MemorySet` 锁内直接调用文件后端，是因为文件 I/O 可能获取页缓存、inode、ext4 和块设备锁，不仅会放大地址空间临界区，也会引入 MM—FS 锁序环。快照对象与 `MemorySet` 脱钩，使后端 I/O 的错误可以在修改映射前返回给 `syscall`，同时保持内存层和文件层的责任边界。进程退出没有可接收 `errno` 的调用方，因此退出路径会记录写回错误后继续释放地址空间，而不会因无法返回错误而泄漏进程资源。

当前实现没有利用硬件 `dirty bit` 做精确脏页追踪，因此会保守写回已驻留、可写的共享文件页。`MS_INVALIDATE` 需要 inode 级的全局失效协议，当前明确返回 `EOPNOTSUPP`；文件截断后访问已映射页的完整 `SIGBUS` 边界也尚未建立。这些限制界定了当前共享写回语义，不影响已实现的 `MS_ASYNC` 页缓存写入和 `MS_SYNC` 同步刷新路径。

4.8 大 ELF 按需装载

RespOS 对文件系统 ELF 采用 `file-backed PT_LOAD` 设计，因为“先把整个可执行文件读入内核 `Vec`，再为用户段分配物理页”会使内核堆同时承受整文件副本和 loader 中间对象。对 45,559,552 字节的 `cargo` 而言，仅临时文件副本就接近 RV64 64 MiB 内核堆的容量。

`try_from_elf_file` 只读取 ELF64 header、program header 和 `PT_INTERP` 名称，并将元数据前缀硬限制为 1 MiB。主程序的 `PT_LOAD` VMA 持有 `Arc<dyn FileOp>`、页对齐偏移和文件有效长度；首次访问才分配私有 `frame` 并按页读取，而新页未被文件覆盖的部分天然保持为零，满足 BSS 语义。

表 4-4 对比了整文件 `eager loader` 与当前 `file-backed loader` 的资源边界。

表 4-4 整文件装载与 `file-backed ELF` 对比

对比项	整文件 eager loader	当前 file-backed loader
exec 临时内存	随 ELF 文件大小增长	只保留有界元数据, 上限 1 MiB
PT_LOAD 驻留	exec 阶段集中分配	首次访问时逐页分配
BSS 处理	loader 显式填充	页帧先清零, 只覆盖文件有效字节
异常 ELF	可能放大内核分配	检查 header 尺寸、偏移、文件末尾和页内同余, 失败返回 ENOEXEC

该路径已支撑上述 cargo 在 RV64 -smp 8 -m 8G 环境中运行到 BUILDSTORM_TOOLCHAIN ok。这一结果证明动态物理内存发现、大文件元数据有界读取和私有 file fault 已能共同支撑大型工具链启动。这不等于 BuildStorm 最终多核编译已通过；当前动态链接器仍使用整文件读取路径，后续可继续复用同一 file backing 抽象。

4.9 多核 TLB 可见性与生命周期

当 CLONE_VM 任务在多个 CPU 上共享同一 MemorySet 时，修改 PTE 后只执行本地 sfence.vma 是不够的，因为远端 hart 可能继续使用旧 TLB 项，甚至在页帧已被回收后仍访问原 PPN。RespOS 因此在 RV64 MemorySet 中维护真实 active_hart_mask：调度器在恢复任务前、持地址空间读锁发布当前 hart bit；只有当 CPU 已切回 per-CPU idle/kernel 页表后才清除。

页表修改者（持 MemorySet 写锁）

- 更新 PTE
- 顺序 fence + 本地 sfence.vma
- active_hart_mask - current_hart
- SBI remote SFENCE 到相关远端 hart
- 当前 OpenSBI 等待远端 fence 完成后返回
- 之后才能安全回收旧页帧

图 4-5 RV64 共享地址空间的定向 TLB shutdown

图 4-5 所示协议的优势是只向当前真正加载该地址空间的远端 hart 发送 RFENCE，避免对所有 CPU 广播。它依赖当前 QEMU/OpenSBI 的同步 RFENCE 完成语义；更换 firmware、绕过 SBI 或引入 ASID 后，都必须重新审计 request/ack 与页帧回收顺序。LA64 当前不沿用这一 RV64 专用验收结论。

地址空间的退出同样遵循真实工作集。MapArea::unmap 对 Direct 映射遍历全范围，对可能巨大而稀疏的 Framed VMA 只遍历 data_frames 中的驻留 VPN；页表拥有的页帧则先进入架构后端的有限隔离队列，避免退出路径仍短暂运行在旧页表时立即重用其页表页。这使回收复杂度与实际驻留内存而不是虚拟保留跨度成比例。

4.10 与其他模块的协作

内存管理不直接决定 syscall ABI、进程调度或文件持久化，而是向这些模块提供可组合的地址空间协议。表 4-5 给出主要边界。

表 4-5 内存管理的跨模块接口

协作模块	内存层提供的能力	对方必须保证的条件
syscall	范围校验、VMA 建图/切分/改权限、用户 copy	只解析 ABI 和 errno, 不绕过 MemorySet 直接修改 PTE
task / scheduler	COW 派生、CLONE_VM 共享、地址空间激活/回收	exec/exit 回收前确保远端 sibling 不再使用旧地址空间
trap / arch	公共 handle_page_fault、页表操作与 TLB 刷新入口	将架构故障归一为 Instruction/Load/Store, 正确实现 PTE/COW 标志
VFS / page cache	file-backed VMA、共享文件页和写回快照	FileOp 提供按偏移读写、stat、fsync 和 mmap 生命周期回调
procfs / sysinfo	真实物理页帧数和内核堆统计	不把早期最大可达窗口当成真实 RAM

4.11 功能与设计成果总结

RespOS 内存管理的主线不是罗列更多分配算法，而是在统一所有权模型上完成 Linux ABI、文件系统和多核的组合。表 4-6 总结已形成的设计成果及其证据边界。

表 4-6 内存管理成果与当前边界

方向	已实现的设计成果	当前证据或边界
地址空间	MemorySet-VMA-PTE-Frame 统一模型, 范围修改共用切分逻辑	debug 不变量与 split self-test 直接检查结构正确性
节省实际内存	私有匿名页、私有文件页和主 ELF 按需驻留; fork 使用 COW	无 swap 和通用页回收器; LA64 brk 当前为 eager
文件共享映射	共享 file frame + 锁外快照写回	2026-08-02 记录的 mmap/munmap 子集中, RV64 musl/glibc 各 20 通过、2 个已知边界失败; LA64 各 15 通过、0 失败; MS_INVALIDATE 和 truncate-SIGBUS 未完整实现
大程序装载	FDT 动态 RAM + file-backed 主 ELF + 1 MiB 元数据上限	2026-08-06 的 RV64 8 核/8 GiB 记录达到 BUILDSTORM_TOOLCHAIN ok; 最终 minibuild/多核编译不由此结果代替
多核页表	RV64 active-hart 跟踪和定向 SBI RFENCE, 旧 frame 回收前完成 shutdown	2026-08-06 的 2/8 核共享 MM 专项压力已通过; 结论限于当前 QEMU/OpenSBI, LA64 SMP 待专项验证

综上, RespOS 已将“分配一页内存”扩展为跨越进程、文件、trap 和多核调度的地址空间协议: 虚拟范围可以延迟承诺, 但权限必须在 VMA 和 PTE 两层一致; 物理页可以共享, 但所有权和回收必须有唯一落点; 写回和页表替换可以分阶段执行, 但失败不能留下半完成状态。

5. 文件系统

本章回答: RespOS 如何把文件描述符、路径、目录项、inode 和具体文件系统连接起来? ext4、procfs、devfs 等后端如何复用同一套接口? 页缓存和写回如何与文件 I/O、内存映射协作?

RespOS 的文件系统参考并实现了类 Linux 的 VFS 分层模型。它首先把三个容易混在一起的问题分开: 文件在目录树中的位置是什么, 文件本身具有什么属性和操作, 某一次打开之后又处于什么状态。路径查找解决第一个问题, InodeOp 解决第二个问题, FileOp 解决第三个问题, FdTable 再把这些内核对象映射为用户态看到的整数 fd。

可以用三句话区分这些层次: Dentry/Path 表示“从哪里找到它”, InodeOp 表示“找到的对象是什么”, FileOp 表示“这次打开怎样使用它”。因此, InodeOp 不保存某个调用者的文件偏移, 也不直接对应用户态 fd; FileOp 才保存一次打开实例的 offset、status flags 和 poll 状态。一个 inode 可以对应多个打开实例, 一个打开实例又可以被多个 fd 共享。

文件系统主要以 ext4 作为持久化后端, 同时接入 procfs、devfs、tmpfile、pipe、socket 等对象。它们不需要都具有磁盘 inode 或路径, 但只要实现相应的 FileOp, 就可以复用 read、write、poll、fcntl 和 close 等文件描述符语义。文件后端 I/O 与地址空间管理则通过 FileOp 的按偏移访问和 mmap 写回接口连接起来。

5.1 VFS 对象模型

5.1.1 File: 一次打开文件的运行时状态

文件系统对象进入用户态前, 会经历“路径找到 inode、根据 inode 建立打开对象、把打开对象放入 fd 表”的过程。RespOS 中, 普通文件的打开实例由 File 表示, 并实现 FileOp trait。File 自身保存稳定的 inode 引用, 动态变化的打开状态则集中放在受互斥锁保护的 FileInner 中:

```
pub struct File {
    inode: Arc<dyn InodeOp>,
    inner: Mutex<FileInner>,
}

struct FileInner {
    offset: usize,
    path: Arc<Path>,
    flags: OpenFlags,
    dirent_cache: Option<Arc<Vec<LinuxDirent64>>>,
    page_cache: Option<Arc<PageCache>>,
    write_back: bool,
    // 省略 tmpfile 与时间覆盖字段
}
```

代码片段 5-1 File 与 FileInner 的核心状态

FileInner 中的 offset 以字节为单位, 表示该 open-file description 当前的访问位置; path 将这次打开与挂载实例中的 dentry 绑定; flags 保存打开状态; page_cache 和 write_back

则把普通文件的 I/O 与缓存写回连接起来。File 构造时会增加 ext4 inode 的打开计数，并根据 inode 类型取得共享页缓存。

打开标志会直接影响初始状态。若指定 `O_TRUNC` 且以写模式打开，构造阶段先将文件截断并同步调整缓存长度；若指定 `O_APPEND`，普通文件的初始 `offset` 设置为当前文件大小，否则从零开始。之后 `read/write` 在 `FileInner` 锁内推进 `offset`，因此两个独立的 `open` 拥有独立访问位置，而 `dup` 或 `fork` 复制同一个 `Arc<dyn FileOp>` 时会共享该位置。

5.1.2 FileOp：统一不同类型文件的使用方式

File 实现 `FileOp`，这个 `trait` 描述用户通过 `fd` 使用一个打开对象时需要的行为，而不是描述磁盘上的文件身份。普通文件、管道、标准输入输出、`socket`、`epoll` 和 `memfd` 都可以以 `Arc<dyn FileOp>` 进入 `FdTable`：

```
pub trait FileOp: Any + Send + Sync {
    fn read(&self, buf: &mut [u8]) -> SysResult<usize>;
    fn write(&self, buf: &[u8]) -> SysResult<usize>;
    fn read_at_offset(&self, offset: usize, buf: &mut [u8]) -> SysResult<usize>;
    fn write_at_offset(&self, offset: usize, buf: &[u8]) -> SysResult<usize>;
    fn can_seek(&self) -> SysResult;
    fn seek(&self, offset: isize) -> SysResult<usize>;
    fn get_offset(&self) -> usize;
    fn set_status_flags(&self, flags: OpenFlags) -> SysResult;
    fn readable(&self) -> bool;
    fn writable(&self) -> bool;
    fn read_ready(&self) -> bool;
    fn write_ready(&self) -> bool;
    fn register_poll_waiter(&self, tid: usize, events: PollEvents) -> bool;
    fn fsync(&self) -> SysResult<usize>;
}
```

代码片段 5-2 FileOp 的核心操作接口

其中，`read`、`write` 和 `seek` 使用并更新打开实例的共享 `offset`；`read_at_offset` 和 `write_at_offset` 则提供不改变 `offset` 的无状态访问，供 `pread`、`pwrite` 和文件映射等路径使用。`read_ready`、`write_ready` 与 `register_poll_waiter` 表达阻塞和就绪语义：普通文件通常直接报告可读写，`pipe` 根据缓冲区和端点状态判断，`socket` 则根据协议缓冲区判断。不可 `seek` 的对象保留默认的 `ESPIPE` 行为，特殊文件可以覆盖相应操作。

`FileOp` 的价值在于把“如何使用文件”从“文件实际存储在哪里”中分离出来。`pipe` 和 `socket` 不需要伪造 ext4 inode，也可以通过同一接口接入 `read`、`write`、`poll` 和 `close`；普通文件则在 `FileOp` 实现内部继续调用 `InodeOp`、`Path` 和 `PageCache`。描述符自身的 `CLOEXEC` 位保存在 `FdEntry` 中，不属于共享的 `FileOp` 状态。

5.1.3 InodeOp：文件对象的后端能力边界

如果说 `FileOp` 回答“这次打开怎么用”，`InodeOp` 回答的就是“这个文件对象是什么、后端能够对它做什么”。它不保存某个打开实例的 `offset`，也不直接对应用户态 `fd`，而是提供节点类型、属性、按偏移 I/O、截断、目录查找、创建、链接、删除和可选的页缓存等能力：

```
pub trait InodeOp: Any + Send + Sync {
    fn as_any(&self) -> &dyn Any;
    fn node_type(&self) -> InodeType;
    fn stat(&self, path: &str) -> SysResult<KStat>;
    fn read_at(&self, path: &str, off: usize, buf: &mut [u8]) -> SysResult<usize>;
}
```

```

fn write_at(&self, path: &str, off: usize, buf: &[u8]) -> SysResult<usize>;
fn truncate(&self, path: &str, size: usize) -> SysResult<usize>;
fn lookup(&self, parent_path: &str, name: &str)
    -> SysResult<Arc<dyn InodeOp>>;
fn readdir(&self, path: &str) -> SysResult<Vec<LinuxDirent64>>;
fn create(&self, parent_path: &str, name: &str, ty: InodeType)
    -> SysResult<Arc<dyn InodeOp>>;
fn link(&self, old_path: &str, dentry: Arc<Dentry>) -> SysResult;
fn unlink(&self, dentry: &Arc<Dentry>) -> SysResult;
}

```

代码片段 5-3 InodeOp 的后端能力接口

`read_at` 和 `write_at` 使用显式偏移，因而同一 `inode` 可以被多个打开实例访问而不会共享打开位置；`lookup`、`create` 和 `unlink` 则表达目录在命名空间中的操作能力。`get_page_cache` 默认返回 `None`，只有支持普通文件缓存的后端覆盖它；时间、权限和扩展属性接口在不适用的后端上返回明确错误或空结果，`procfs`、`devfs` 等对象不需要伪造磁盘文件语义。

当前接口中的 `path` 参数来自 `lwext4` 的路径型访问方式。VFS 通过 `Path` 和 `Dentry::current_abs_path()` 生成当前后端路径，再由具体 `inode` 实现把统一操作转换成底层调用。这个参数是当前 `ext4` 适配策略的实现细节，不改变 `InodeOp` 的职责：它仍然表示文件对象的后端能力，而不是一次打开的状态。

5.1.4 Ext4Inode: InodeOp 的具体实现

`Ext4Inode` 是 RespOS 对 `ext4` 普通文件和目录的 VFS 适配对象。它没有把 `lwext4` 的临时打开句柄直接暴露给上层，而是长期保存 `inode` 身份、路径变化和内核侧缓存状态；每次执行具体 I/O 时，再以当前有效路径创建短生命周期的 `Ext4File` 调用 `lwext4`：

```

pub struct Ext4Inode {
    pub ino: u64,
    ty: Ext4InodeTypes,
    times: Mutex<Option<InodeTimes>>,
    mode_override: Mutex<Option<u32>>,
    owner_override: Mutex<Option<(u32, u32)>>,
    nlink_override: Mutex<Option<u32>>,
    orphan_path: Mutex<Option<String>>,
    renamed_path: Mutex<Option<String>>,
    open_files: AtomicUsize,
    page_cache: Arc<PageCache>,
    xattrs: Mutex<HashMap<String, Vec<u8>>>,
}

```

代码片段 5-4 Ext4Inode 的核心状态

这些字段分别承担四类职责：`ino` 和 `ty` 标识后端对象；时间、权限、所有者和链接数覆盖字段保存 VFS 需要但当前后端不能直接稳定提供的元数据；`orphan_path` 与 `renamed_path` 处理路径变化期间仍然存活的 `inode`；`open_files` 区分真正的打开实例数量；`page_cache` 使同一个 `inode` 的多个 `File` 共享文件数据缓存。

`ext4 inode` 的创建经过 `inode cache`：`get_or_create` 先按 `inode` 号查找全局缓存，缓存中只保存 `Weak` 引用；已有 `Arc<Ext4Inode>` 仍被使用时，新的路径查找会复用同一 `inode` 对象，所有打开实例也就共享同一份页缓存和内核侧元数据。缓存项不拥有 `inode` 的生命周期，最后一个强引用释放后，失效的弱引用可以被清理，避免缓存本身阻止 `inode` 回收。

具体操作需要同时处理 VFS 语义和 `lwext4` 的错误、锁与路径约束。`lookup` 通过 `ext4` 目录遍历获得子 `inode` 号和类型，再调用 `get_or_create`；`read_at`、`write_at`、`stat` 和 `truncate` 使用 `storage_path` 选择当前后端路径，创建临时 `Ext4File` 完成底层操作，并通过 `map_lwext4_err` 把 `lwext4` 错误转换为 Linux 风格 `errno`。对共享的 `lwext4` 入口，适配层使用 `EXT4_OP_LOCK` 串行化 FFI 调用，避免底层全局状态被并发破坏。

`Ext4Inode` 还承担普通 `inode` 生命周期中最难由单次路径调用解决的两类语义：

- `rename`。`dentry` 可以保持自身身份并更新父链，但 `lwext4` 后端需要访问新的路径。RespOS 将新路径记录到 `renamed_path`，使已经打开的 `File` 后续通过 `storage_path` 不再访问旧路径。
- `unlink` 与延迟清理。如果文件仍有打开者且链接数归零，适配层先把后端文件改名为 `.respos_orphan_<ino>`，将内核侧链接数覆盖为零；最后一个 `File` 关闭时，`open_files` 归零并删除这个隐藏路径。这样目录项可以先从命名空间消失，已有 `fd` 仍然可以继续访问文件内容。

因此，`Ext4Inode` 并不是简单的“`lwext4 inode` 指针包装”，而是 VFS 语义与路径型 `ext4` 后端之间的适配层：它统一缓存 `inode` 身份，维护打开计数和元数据覆盖，转换错误码，协调 `rename/unlink` 生命周期，并把普通文件读写接入共享页缓存。

5.1.5 Dentry、Path 与对象定位

`Path` 是 (`VfsMount`, `Dentry`) 二元组，表示一个对象在整个挂载命名空间中的位置；`Dentry` 维护名字、父目录、子目录缓存以及可选的 `Arc<dyn Inode0p>`：

```
pub struct DentryInner {
    pub inode: Option<Arc<dyn Inode0p>>,
    pub parent: Option<Arc<Dentry>>,
    pub children: HashMap<String, Weak<Dentry>>,
    alias_path: Option<String>,
}
```

代码片段 5-5 `Dentry` 的路径树状态

`inode == None` 表示负目录项，可缓存“该名字不存在”的结果。父节点使用强引用以支持沿父链重建路径，子节点使用 `Weak` 避免形成永久引用环。`rename` 时 `dentry` 保持对象身份，通过父链和 `alias_path` 计算新的后端路径；`unlink` 删除名字空间中的 `dentry` 连接，但不必立即销毁仍被打开的 `inode`。

5.2 挂载树与路径解析

5.2.1 VfsMount 与 Mount

RespOS 将“一个文件系统实例”和“它挂载到哪里”分开表示。`VfsMount` 保存该实例的根 `dentry`、超级块和挂载 `flags`；`Mount` 保存挂载点、父挂载和子挂载，形成全局 `mount tree`：

```
pub struct VfsMount {
    pub root: Arc<Dentry>,
    pub fs: Arc<dyn SuperBlock0p>,
    flags: AtomicI32,
}

pub struct Mount {
    pub mountpoint: Arc<Dentry>,
    pub vfs_mount: Arc<VfsMount>,
}
```



```
pub parent: Option<Weak<Mount>>,
pub children: Mutex<Vec<Arc<Mount>>>,
}
```

代码片段 5-6 文件系统实例与挂载树节点

路径解析遇到挂载点时，会把游标中的 `mnt` 和 `dentry` 一起切换到子文件系统根目录。反向解析 `..` 时，如果当前 `dentry` 已经是子文件系统根，则沿 `Mount.parent` 返回父文件系统中的挂载点。这样，路径语义既能在每个文件系统内部保持独立，又能在全局命名空间中正确跨越挂载边界。

当前初始化流程把 `ext4` 根文件系统作为主存储后端，再将 `procfs` 和 `devfs` 以独立的 `VfsMount` 挂载到 `/proc` 和 `/dev`。`tmpfs/shm` 等接口也通过 `VFS` 和特殊文件对象提供相应语义。部分挂载使用根文件系统中的隐藏 `backing` 目录保存后端数据，以便在现有 `ext4` 镜像上提供隔离的挂载根；这属于当前实现的适配方案，不应误解为每个虚拟文件系统都直接拥有独立块设备。

5.2.2 namei 路径遍历

路径查找由 `Nameidata` 驱动。它保存当前挂载实例、当前 `dentry`、待处理的路径分量和遍历深度。相对路径从当前工作目录或 `dirfd` 对应的目录开始，绝对路径从任务 `root` 开始；`dirfd`、空路径、符号链接跟随等路径操作选项由 `syscall` 层传入并在 `namei` 中解释。本章只展开这些选项如何改变查找起点和遍历过程，不逐项罗列所有 `ABI` 常量。

一次典型的 `link_path_walk` 过程如下：

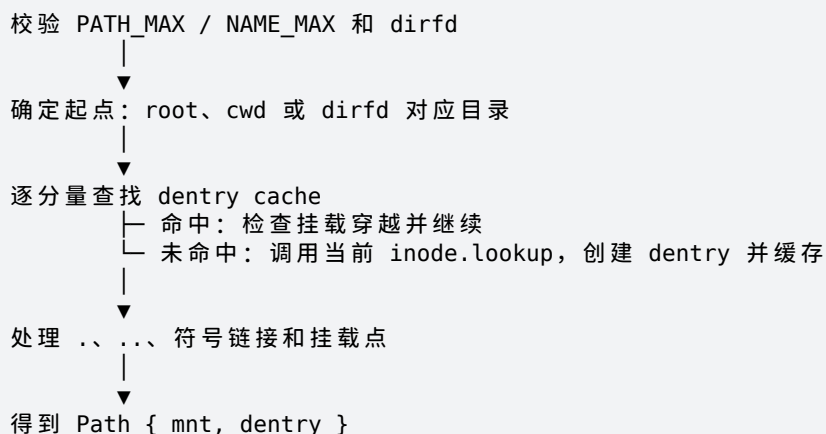


图 5-2 namei 路径解析流程

符号链接展开采用重新组织剩余路径的迭代流程，并限制最多跟随 40 次，避免深层链接造成无限循环或内核栈增长。对 `create`、`unlink`、`rename`、`link` 和 `symlink` 等修改命名空间的操作，当前实现使用 `NAMEI_MUTATION_LOCK` 串行化关键路径，以简化 `dentry cache`、后端路径和失败回滚之间的一致性维护。

这一设计将路径修改、后端操作和 `dentry` 缓存维护纳入同一条可控的更新路径，便于保证 `create`、`unlink` 和 `rename` 的可观察结果一致。它也是当前 `VFS` 适配路径型 `lwext4` 接口时采用的工程取舍：先保证命名空间状态和失败回滚的正确性，再根据实际并发负载进一步细化锁粒度。

5.3 ext4 与虚拟文件系统后端

5.3.1 ext4 适配层

RespOS 通过 `lwext4_rust` 的 FFI 接入 ext4, `Ext4Inode` 负责把 `InodeOp` 的统一语义翻译为 `lwext4` 的路径型接口。VFS 层不直接操作 `lwext4` 的临时文件句柄, 而是由 `inode` 根据当前 `storage_path` 创建短生命周期的 `Ext4File`, 完成读写、属性查询、目录遍历和命名空间修改; 底层错误再由适配层转换为 Linux 风格 `errno`。

这一适配层还补齐了 `lwext4` 不直接替 VFS 管理的语义: `open_files` 单独记录 `File` 数量, 不能用 `Arc::strong_count` 代替; 普通 ext4 `inode` 通过 `get_page_cache` 接入共享页缓存; `rename` 后通过 `renamed_path` 保证旧的打开对象继续访问正确的后端路径; `unlink` 后通过 `orphan` 路径延迟清理, 使名字从目录树中消失后, 已有 `fd` 仍能继续访问文件。具体的 `inode` 状态和这些生命周期协议见 5.1.4, 本节只说明它们如何作为 ext4 后端能力对外提供。

5.3.2 procfs、devfs 与特殊文件

`procfs` 和 `devfs` 都实现 `SuperBlockOp`, 并通过独立挂载实例提供自己的根 `inode`。它们的 `inode` 通常动态生成内容, 而不是从 ext4 数据块读取。例如 `/proc/cpuinfo`、`/proc/meminfo`、`/proc/self/maps` 等文件在 `read` 时根据内核状态生成文本; `/dev/null`、`/dev/zero`、`/dev/random`、`tty`、`shm` 等设备则由各自的 `FileOp` 或 `InodeOp` 实现读写语义。

`pipe` 不依赖磁盘 `inode`。两个端点共享一个带有读写关闭状态、FIFO 等待队列和 `poll waiter` 的环形缓冲区; 当没有读者或写者时, 端点生命周期决定 `EOF` 和 `SIGPIPE` 等可观察行为。`socket`、`eventfd`、`epoll`、`timerfd` 等对象也直接以 `FileOp` 形式进入 `FdTable`。这些对象复用文件描述符、`poll` 和 `close` 机制, 但不应被强行套入 ext4 的页缓存或路径生命周期。

5.4 页缓存与持久化写回

5.4.1 PageCache 的职责

`PageCache` 以页为单位缓存普通文件内容, 并维护缓存长度、页内容、脏状态和写回版本等信息。`File` 构造时从 `inode` 获取共享页缓存; 顺序读、指定偏移读和普通写都优先通过 `PageCache` 完成。首次读取尚未驻留的区间时, 缓存向后端发起按偏移读取; 写入则先修改缓存页, 再根据同步标志或显式同步请求执行写回。

页缓存把用户可见的文件长度和后端磁盘长度分开管理: 写入扩大缓存后, `stat` 可以观察到新的长度, 真正的后端更新由写回链路提交。`O_SYNC`、`O_DSYNC` 和 `fsync` 会提高写回强度; 普通 `close` 路径也会尝试刷新自身的脏页, 最后一个 ext4 打开对象关闭时还会触发孤儿文件清理。

5.4.2 三层写回链

当前普通文件的持久化路径可以概括为:

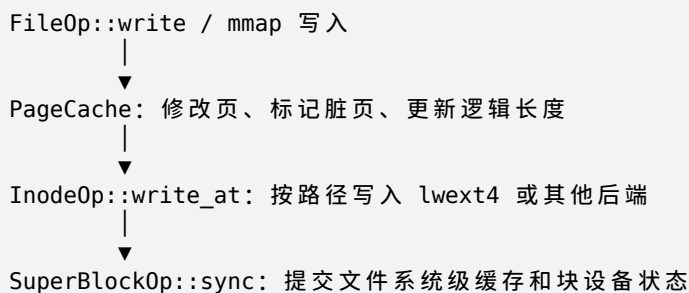


图 5-3 文件写入与持久化写回链路

`File::fsync` 会先刷新关联页缓存，再调用当前路径所属 `VfsMount` 的超级块同步操作。对普通 `ext4` 文件，缓存写回和后端同步是两个层次：前者把内存中的脏页交给 `inode` 后端，后者推动文件系统和设备完成更底层的提交。不能只调用其中一层就宣称数据已经持久化。

5.4.3 与 `mmap(MAP_SHARED)` 的协作

文件映射的写回由 MM 和 FS 共同完成。建立可写共享映射前，`FileOp::mmap_allowed` 检查文件是否可读，并要求 `MAP_SHARED|PROT_WRITE` 对应的 `open-file description` 可写。映射建立后，页面内容由地址空间管理；在 `msync`、`munmap`、`MAP_FIXED` 替换、`mremap` 收缩/覆盖、`mprotect` 或进程退出等路径中，MM 在地址空间锁内取得 `resident frame` 的快照，然后释放 `MemorySet` 锁，再通过 `FileOp` 写入页缓存或后端，最后按需要调用 `fsync`。

这种“快照—锁外 I/O—同步”的协议有两个目的：避免文件后端 I/O 持有地址空间写锁导致锁序反转，也避免后端阻塞把所有地址空间操作串在一起。由于当前硬件路径没有统一提供 `dirty bit`，系统对 `resident writable shared file pages` 采用保守写回，从而以更明确的同步行为换取实现可靠性。

5.5 文件描述符表与生命周期

5.5.1 `FdTable` 与 `FdEntry`

`FdTable` 是进程持有的描述符槽位集合，内部使用自旋锁保护槽位数组，并维护下一个分配位置及 `RLIMIT_NOFILE` 限制。每个槽位保存一个 `FdEntry`：

```
pub struct FdEntry {
    pub file: Arc<dyn FileOp>,
    pub flags: OpenFlags,
}
```

代码片段 5-7 描述符槽位与打开对象的关系

`FdEntry::flags` 保存每个描述符独有的标志，当前最重要的是 `close-on-exec`。`file` 指向共享的 `FileOp`，其内部保存 `offset`、追加写和非阻塞等 `open-file` 状态。关闭描述符时，先从表中移除槽位，再由 `Arc` 引用计数决定何时释放 `FileOp`；普通 `File` 的析构会尝试同步页缓存，并减少 `ext4 inode` 的打开计数。

5.5.2 `dup`、`fork`、`exec` 与 `close`

- `dup` 和 `fork` 复制 `Arc<dyn FileOp>`，因此共享文件偏移和打开状态，但为新描述符创建独立的 `FdEntry`，可以单独设置 `close-on-exec`。
- `exec` 根据每个 `FdEntry` 的 `close-on-exec` 状态关闭描述符；共享的 `FileOp` 只有在没有其他引用时才析构。
- `close` 从 `FdTable` 移除槽位后，`pipe`、`socket`、特殊 `fd` 和普通文件分别执行自己的最后引用清理；`pipe` 的最后一个写端关闭才会让读端观察到 EOF，最后一个读端关闭则影响写端错误和阻塞唤醒。
- `fcntl` 修改描述符标志和打开状态时必须区分上述两个对象，不能把 `close-on-exec` 写入 `File` 的共享状态。

这一分层也解释了为什么文件系统章节不能只讨论磁盘 `inode`：用户态可观察的文件语义同时由命名空间身份、打开实例状态和描述符生命周期决定。

5.6 功能与设计总结

文件系统并不是孤立的存储模块，而是连接系统调用、进程、内存和设备的基础设施。表 5-2 总结了 RespOS 形成的主要功能和对应设计。

功能或设计	主要实现	——	——	——	VFS 对象抽象	通过 Path、Dentry、InodeOp、FileOp 和 SuperBlockOp 分离路径、命名、文件对象、打开状态与后端存储	磁盘文件系统	通过 lwext4_rust 接入 ext4，支持普通文件、目录、符号链接、硬链接、rename、unlink 和属性操作	虚拟文件系统	以独立挂载实例接入 procfs、devfs，并提供 /proc、/dev、shm 等内核和设备接口	路径与挂载	支持 cwd/root/dirfd 起点、dentry cache、符号链接、./.. 以及跨挂载点路径遍历	文件访问性能	通过 ext4 inode 级共享页缓存、目录项缓存和目录读取缓存减少后端访问	数据一致性	通过页缓存写回、fsync、unlink 孤儿文件和 rename 回滚维护文件数据与命名空间的一致性	内核模块协作	通过 file-backed VMA、mmap 写回、pipe、socket、poll 和统一 fd 生命周期连接 MM、task、IPC、network 与 drivers
-------	------	----	----	----	----------	---	--------	---	--------	--	-------	--	--------	---	-------	---	--------	---

表 5-2 文件系统主要功能与设计对应关系

从整体上看，RespOS 文件系统形成了从用户态 fd 到 ext4 块设备的完整调用链：fd 首先定位到打开对象，打开对象持有路径，路径连接挂载实例和 dentry，dentry 再通过 InodeOp 调用具体文件系统。围绕这条主线，系统同时支持磁盘文件、虚拟文件、设备文件和无路径特殊文件，并将缓存、同步、内存映射和进程生命周期纳入统一的文件对象模型。

这种设计既保持了 Linux VFS 模型对用户态的可理解性，又利用 Rust 的 Arc、trait 和错误传播机制管理了对象生命周期、后端替换和失败路径，为后续扩展更多文件系统或设备类型保留了清晰接口。

6. 进程间通信与信号

本章回答：RespOS 如何在异步通知、字节流传输、用户态同步和共享地址空间之间建立统一而可中断的进程协作机制？

6.1 概述与机制分工

RespOS 没有把进程间通信实现为单一总线，而是按通信语义选择不同机制：信号传递离散异步事件，管道传递有序字节流，`futex` 只在竞争发生时进入内核，`System V` 共享内存让多个地址空间直接映射同一组物理页。四者最终都与任务调度器协作，因此“如何登记等待者、由谁完成事件、何时唤醒任务”是本章共同的正确性主线。

表 6-1 对比了四种机制的对象身份、阻塞条件和典型用途。它们并非相互替代，而是分别覆盖通知、传输、同步和大块共享四类需求。

表 6-1 RespOS IPC 与信号机制分工

机制	内核中的核心对象	是否传输数据	何时进入阻塞	典型用途
信号	每线程 <code>SigPending</code> 、线程组共享 <code>SigHandler</code>	<code>SigInfo</code> 与信号编号	<code>sigsuspend</code> 、 <code>sigtimedwait</code> 或打断其他可中断等待	异步通知、timer、异常、子进程状态变化
管道	共享 <code>PipeRingBuffer</code> 与读写端 <code>FileOp</code>	有序字节流	读空且仍有写端；写满且仍有读端	shell 重定向、父子进程输出捕获
<code>futex</code>	<code>FutexKey</code> 、哈希等待队列、完成状态	用户数据留在共享内存，内核只管理等待关系	用户值等于期望值且发生竞争	pthread <code>mutex</code> 、 <code>condvar</code> 、线程 <code>join</code>
<code>System V shm</code>	<code>ShmSegment</code> 、共享 frame、每次 <code>attach</code> 记录	进程直接读写同一物理页	映射本身不阻塞	大块共享数据、跨进程共享状态

6.2 信号机制

6.2.1 线程状态与进程共享状态

RespOS 将信号状态分为线程私有和线程组共享两层。每个线程独立持有 `pending` 位图、`mask`、每个信号号对应的一份 `SigInfo` 以及备用信号栈；同一线程组共享 `handler` 表。这一划分使 `sigprocmask`、`sigsuspend` 和 `sigaltstack` 保持线程语义，同时保证任一线程调用 `sigaction` 后，整个进程看到一致的处理规则。

代码片段 6-1 给出了 `pending` 状态的当前表示。位图负责快速查找最小的可投递信号，`BTreeMap` 保存与该信号号关联的信息；同一个信号号再次到达会覆盖 `info` 并保持一个 `pending` 位，因此当前语义不是 Linux 实时信号的逐项排队模型。

代码片段 6-1 每线程信号挂起状态

```
// os/src/signal/sig_struct.rs
pub struct SigPending {
    pub pending: SigSet,
    pub mask: SigSet,
    pub info: BTreeMap<i32, SigInfo>,
}
```

表 6-2 进一步说明了各对象的创建、共享和释放边界。exec 会清空 pending、重置备用栈，并将用户自定义 handler 恢复为默认动作；显式设为忽略的 handler 保持忽略，这与“新映像不继承旧用户函数地址，但保留忽略选择”的语义相符。

表 6-2 信号对象的生命周期与可见范围

状态	共享范围	主要更新者	生命周期边界
pending + SigInfo	每线程	kill/tkill/tgkill、异常、timer、pipe、子进程事件	投递或 sigtimedwait 消费；exec 清空
signal mask	每线程	sigprocmask、sigsuspend、handler 进入与 sigreturn	clone 继承当前 mask；sigreturn 恢复旧 mask
alt stack	每线程	sigaltstack	新线程使用默认值；exec 重置
handler 表	线程组共享	sigaction、SA_RESETHAND、exec	同组线程共同可见；exec 重置用户 handler

6.2.2 从产生到 sigreturn 的完整链路

信号来源包括 kill/tkill/tgkill、用户缺页或非法访问、timer 到期、向无读端管道写入以及子进程停止/继续/退出。线程级信号直接进入指定 tid；进程级信号在目标线程组中选择一个当前未屏蔽该信号的线程，若暂时没有成员可立即接收，则回退到 leader 或组内成员保存 pending。kill 还按 uid/euid/suid 检查发送权限，tgkill 同时校验 tid 是否属于指定 tgid。

如图 6-1 所示，内核并不在任意指令位置直接跳入用户 handler，而是在 trap 返回用户态前统一调用 handle_signal()。这时完整寄存器现场已经位于任务内核栈，内核可以在用户普通栈或备用栈上构造 signal frame，再通过修改 trap context 改写返回 PC、SP 和参数寄存器。

图 6-1 信号产生、挂起、投递与返回流程

```
信号源
├─> 选择目标 tid
│   └─> pending 位图 + SigInfo
│       ├──> 目标处于可中断等待：争取 Interrupted 并唤醒
│       └─> 目标继续运行
│           └─> trap 返回用户态前 handle_signal()
│               ├──> 默认动作：ignore / stop / continue / terminate
│               └─> 用户 handler：构造 SigFrame 或 SigRTFrame
│                   └─> handler 返回 trampoline
│                       └─> sigreturn 恢复寄存器、PC 与旧 mask
```

普通 handler 使用 SigFrame 保存 SigContext 和旧 mask；设置 SA_SIGINFO 时使用 SigRTFrame，并向 handler 传入 siginfo_t 与 ucontext。SA_ONSTACK 选择备用栈，SA_NODEFER 决定 handler 执行期间是否自动屏蔽当前信号，SA_RESETHAND 在首次投递后恢复默认动作。所有 frame 都通过受检用户拷贝写入；构造失败时不会带着半成品现场返回用户态，而是按致命信号终止当前任务。

默认动作按表 6-3 分为四类。Core 类当前与 Term 一样终止线程组，但不会生成 core 文件；Stop 和 Continue 会记录 wait event 并向父进程发送 SIGCHLD，使 `wait4(WUNTRACED/WCONTINUED)` 能观察到状态变化。

表 6-3 默认信号动作与当前行为

动作类别	代表信号	RespOS 行为
Ignore	SIGCHLD、SIGURG、SIGWINCH	消费后不改变任务状态
Stop	SIGSTOP、SIGTSTP、SIGTTIN、SIGTTOU	任务进入 Stopped ，父进程可等待该事件
Continue	SIGCONT	将 stopped 任务重新发布为 Ready ，并记录 continued 事件
Term / Core	SIGTERM、SIGKILL / SIGSEGV、SIGABRT 等	进入线程组退出；Core 类暂不写 core 文件

6.2.3 可中断等待与架构安全约束

信号投递与调度器之间采用“pending + 可中断标志 + 具体等待队列清理”的协作方式。目标线程进入 `futex`、`pipe`、`wait4` 或 `signal wait` 前先设置 `interruptible`；投递者发现存在未屏蔽且非忽略信号后，将完成状态改为 **Interrupted**，从 `futex` 等专用队列移除 waiter，并尝试从 scheduler 的 `blocked` 集合唤醒 tid。等待者恢复后返回 **EINTR**，而不是继续无期限睡眠。

这里的关键不是多调用一次 `wakeup_task()`，而是让 `wake`、`timeout`、`signal` 和 `exit` 只能有一个完成者。否则同一 tid 可能重复进入 ready queue，或者一条路径删除了另一条路径仍要使用的 waiter。`futex` 将这一规则显式编码为完成状态；`pipe` 和 `wait` 则通过“锁内登记 + 发布 `blocked` 后复查 + 恢复时清理残留”关闭竞态窗口。

RV64 的 `signal return` 还遵守统一的 trap 安全顺序：写回 `sstatus` 前清除 SIE，保持 kernel trap vector 直到通用寄存器和 per-CPU `sscratch` 恢复完成，最后才切换 user vector 并执行 `sret`。signal frame 来自用户地址空间，不能信任其中保存的中断使能位；这一约束同时保护普通 `syscall` 返回、handler 首次进入和 `sigreturn`。

当前 `SA_RESTART` 只完成 flag 解析，尚未实现系统调用自动重启，被信号打断的可中断系统调用通常返回 **EINTR**。pending 又按信号号合并，因此同一种实时信号连续到达时只保留一个 pending 位和一份 `SigInfo`。这两个边界明确限定了当前信号兼容层的范围。

6.3 管道：字节流、阻塞与端点生命周期

6.3.1 核心对象

匿名管道由两个 `Pipe` 端点共享一个 `PipeRingBuffer`。每个端点实现 `FileOp`，读端只允许 `read`，写端只允许 `write`；`pipe2` 将两个端点分别安装进 fd table，并在第二个 fd 分配或最终 copyout 失败时回滚已经分配的描述符。代码片段 6-2 展示了缓冲区中与同步直接相关的状态。

代码片段 6-2 管道环形缓冲区与等待队列

```
// os/src/fs/pipe.rs
struct PipeRingBuffer {
    buffer: VecDeque<u8>,
    capacity: usize,
    read_closed: bool,
    write_closed: bool,
    read_waiters: VecDeque<usize>,
    write_waiters: VecDeque<usize>,
}
```

```
poll_waiters: Arc<PollWaiters>,
}
```

缓冲区默认容量在 RV64 与 LA64 上均为 64 KiB, 并可通过受限接口按页调整。read/write waiter 保存 tid, poll waiter 则独立保存关注的事件掩码: 前者负责把真正阻塞的任务移回 ready queue, 后者负责通知 poll/epoll 重新检查文件状态。

6.3.2 读写阻塞协议

表 6-4 给出了读写端在不同缓冲区和端点状态下的行为。阻塞 read/write 都是可中断等待, 信号到达后返回 EINTR; O_NONBLOCK 则在无法立即推进且尚未传输数据时返回 EAGAIN。

表 6-4 管道读写状态机

操作与条件	返回或状态变化	唤醒对象
read 且缓冲区非空	取出可用字节并返回	唤醒一个 writer, 通知 POLL_WRITE
read、缓冲区空、仍有写端	reader 登记并进入 Blocked	等待 write、写端关闭或 signal
read、缓冲区空、所有写端关闭	返回 0 (EOF)	poll 可观察 HUP/可读结束状态
write 且有空间、仍有读端	写入可容纳字节并返回	唤醒一个 reader, 通知 POLL_READ
write、缓冲区满、仍有读端	writer 登记并进入 Blocked	等待读取、读端关闭或 signal
write 且所有读端关闭	返回 EPIPE, sys_write 产生 SIGPIPE	当前任务按信号规则处理

管道把“检查条件、登记 waiter、发布 blocked”放在同一把 buffer 锁保护的区间内, 如图 6-2 所示。写端必须取得同一把锁才能写入并取出 reader tid, 因此不会在 reader 检查完空缓冲区、尚未登记时丢掉唤醒。若 waker 在 reader 真正 switch_to_next_task() 前已经把它改为 Ready, reader 会撤销 ready 入队并继续当前上下文, 避免不必要的切换。

图 6-2 管道读空时的无丢失唤醒协议

```
reader (持 buffer 锁)
  检查为空且 write_closed = false
  ↳ read_waiters.push(tid)
  ↳ prepare_current_task_blocked()
  ↳ 释放 buffer 锁

writer (取得同一 buffer 锁)
  写入数据 → pop_read_waiter() → wakeup_task(tid)

reader 切换前复查:
  已经 Ready → 撤销队列项并继续
  仍为 Blocked → switch_to_next_task()
```

6.3.3 EOF、EPIPE 与 poll 可见性

匿名管道没有单独维护“fd 数量计数器”, 其端点生命周期由 Arc<dyn FileOp> 管理。dup 或 fork 让多个 FdEntry 共享同一个端点对象, 只有最后一个写端引用释放时, Pipe::drop() 才设置 write_closed 并唤醒全部 reader; 最后一个读端释放时同理设置 read_closed 并唤醒 writer。因此 EOF 的条件是“缓冲区已空且最后一个写端对象已释放”, 而不是“某个子进程已经 exit”或“当前暂时没有数据”。

这一语义与 fd table 生命周期直接相关。exec 必须先解除 CLONE_FILES 共享再执行 CLOEXEC，退出任务又要在安全时机释放遗留 fd 引用，否则捕获 stdout/stderr 的父进程会一直等不到 EOF。RespOS 的 unshare_fd_table_for_exec() 与 idle loop 的 cleanup_dead_tasks() 正是为这一跨模块引用链提供收敛点。

read_ready() 在“缓冲区非空或写端已关闭”时成立；write_ready() 当前要求至少有一页（或容量本身，若更小）的可写空间，而不是仅检查“未滿”。端点关闭会通知 POLL_READ | POLL_WRITE | POLL_HUP，poll/epoll 随后重新读取实际状态。这使管道的阻塞、非阻塞和事件监控共享同一组状态来源。

6.4 Futex：用户态快路径与内核竞争路径

6.4.1 等待对象与 key

futex 的常见无竞争路径完全在用户态完成，只有用户值表明需要等待时才进入内核。RespOS 使用 256 个哈希桶保存 FutexQ，每个条目包含 FutexKey、等待 tid 和 bitset。key 必须表达“哪些地址实际上代表同一个同步字”，而不能只比较进程内虚拟地址。

表 6-5 给出了当前 key 生成规则。私有 futex 以 tgid 隔离同一虚拟地址；能识别的共享映射则转化为 backing 身份、页索引和页内偏移，使不同进程中的不同虚拟地址仍可落到同一等待队列。

表 6-5 FutexKey 的身份规则

映射类型	scope 来源	key 中的地址部分	目的或边界
FUTEX_PRIVATE_FLAG	tgid	原用户虚拟地址	仅同一线程组内匹配
共享文件映射	文件 dev/ino 派生身份	文件页索引 + 页内偏移	不同进程映射同一文件位置可互相唤醒
已有共享匿名 frame	物理 frame 身份	VMA 页索引 + 页内偏移	fork 继承或共享 frame 映射可匹配
System V shm	当前 attach id	attach 内页索引 + 页内偏移	同一次 attach 内稳定；不同进程独立 shmat 尚不能保证相同
无法解析为共享 backing	tgid	原用户虚拟地址	安全回退为进程内匹配

6.4.2 single-winner 完成协议

一次 futex wait 可能被 wake、deadline、signal 或任务退出结束。代码片段 6-3 中的完成状态把竞争结果显式记录在每个 tid 的 FutexWait 中；状态只能从 Pending 成功改变一次，后到事件发现已经完成后不再重复唤醒。

代码片段 6-3 futex 等待完成状态

```
// os/src/task/futex/wait.rs
enum WaitCompletion {
    Pending,
    Woken,
    TimedOut,
    Interrupted,
}

struct FutexWait {
```



```

deadline: Option<FutexDeadline>,
completion: WaitCompletion,
}

```

表 6-6 展示了每个完成者的提交动作。队列记录、完成记录和 deadline 索引都要同步清理；退出路径虽然不再返回用户态，仍必须删除三处状态，防止悬空 tid 被后续 wake 命中。

表 6-6 futex wait 的竞争完成者

完成者	目标状态	队列与调度动作	用户态结果
FUTEX_WAKE / WAKE_BITSET	Woken	从哈希桶移除并唤醒 tid	返回 0
deadline	TimedOut	删除 queue/deadline 记录并唤醒	ETIMEDOUT
signal	Interrupted	删除 futex waiter, 由 signal 路径唤醒 scheduler	EINTR
task exit	不再恢复	清理 queue、wait、deadline 三处记录	无返回

6.4.3 requeue 的线性化与当前边界

FUTEX_CMP_REQUEUE 要求“比较源 futex 值”和“唤醒/迁移 waiter”属于同一个线性化区间。如果比较发生在队列锁外，另一个线程可能在比较后修改用户值，而内核仍按旧条件迁移 waiter。RespOS 先在锁外调用 check_user_readable() 解析 lazy 页，再在 FUTEX_QUEUES 锁内用固定 4 字节的 read_user_u32_nofault() 复核，随后完成 wake/requeue。这样既把比较与队列修改放在同一临界区，又避免持有 no-IRQ 自旋锁时触发补页或分配。

普通和定时 FUTEX_WAIT 的最终值复核目前仍在持有 FUTEX_QUEUES 时调用通用 copy_from_user(), lazy/COW 页可能进入补页路径；这一点尚未完全收敛到 cmp-requeue 的“锁外预触页、锁内 no-fault 读取”模式。另一个边界来自表 6-5：两个进程分别 shmat 同一个 System V 段时会得到不同 attach id，因此共享段上的跨进程 futex 还缺少稳定的 segment/frame 级身份。

6.5 System V 共享内存

6.5.1 段对象与映射生命周期

System V 共享内存由全局 SHM_TABLE 管理。ShmSegment 保存 key、权限、创建者/最后操作者、时间戳、删除/锁定状态以及一组 Arc<FrameTracker>；每次 shmat 还分配 attach id，将同一组 frame 映射进调用者的 MemorySet。因此，共享的对象身份是 segment/frame，attach 则表示某个地址空间中的一次映射关系。

图 6-3 展示了段从创建到删除的完整生命周期。IPC_RMID 在仍有 attach 时只设置 marked_removed，使新查找不再获得该段；最后一个 attach 消失后才从全局表删除并释放 frames，符合“删除名字不立即破坏现有映射”的延迟删除语义。

图 6-3 System V 共享内存生命周期

```

shmget(key, size)
└─> 创建 ShmSegment + 分配共享 frames
    └─> shmat ─> 分配 attach_id ─> MemorySet 映射同一组 frames
        └─> shmdt: 删除本次 attach
    └─> shmctl(IPC_RMID)
        └─ nattach == 0: 立即删除 segment

```

```
└─ nattach > 0 : marked_removed
   └─> 最后一次 shmdt 后删除 segment
```

表 6-7 总结了各系统调用的对象变化。`SHM_RDONLY`、`SHM_EXEC`、`SHM_RND` 和 `SHM_REMAP` 会影响映射权限或位置，`shmctl` 还提供 `IPC_STAT/SET`、`SHM_STAT/INFO` 及 `LOCK/UNLOCK` 的受限状态管理。

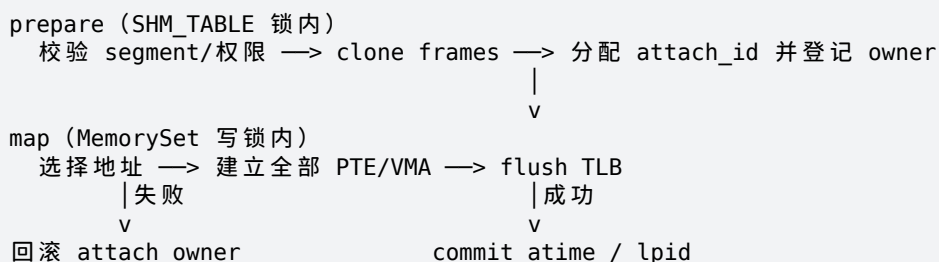
表 6-7 System V 共享内存操作与状态变化

操作	主要检查	成功后的状态变化
<code>shmget</code>	key/ <code>IPC_CREAT/EXCL</code> 、大小、权限、全局页数与段数上限	创建或返回 <code>ShmSegment</code> ，新段一次性分配所需 frames
<code>shmat</code>	shmid、访问权限、地址对齐与已知 flag	在当前 <code>MemorySet</code> 映射共享 frames，登记 attach owner，更新 atime/lpid
<code>shmdt</code>	地址对齐且属于某个 shm attach	删除该 attach，更新 dtime/lpid，必要时完成延迟删除
<code>shmctl(IPC_RMID)</code>	owner 或特权权限	无 attach 时删除；否则标记 <code>SHM_DEST</code>
<code>shmctl(SHM_LOCK/UNLOCK)</code>	owner 或特权权限	记录 locked 状态和 ctime

6.5.2 shmat 的失败原子性

`shmat` 同时修改全局段表和进程地址空间，任一阶段失败都不能留下“统计显示已 attach、页表却没有映射”的半成品。RespOS 先在 `SHM_TABLE` 中校验权限、复制 frame 引用并预留 attach id，然后调用 `MemorySet::mmap_area()` 建图；映射失败会删除 `attach_owners[attach_id]`，只有映射成功后才更新 segment 的 atime 与 lpid。图 6-4 给出了这一 prepare/commit 顺序。

图 6-4 shmat 的 prepare/commit 协议



当前实现不支持 System V 消息队列和 semaphore，`SHM_HUGETLB` 明确返回错误。系统也没有 swap，因此 `SHM_LOCK/SHM_UNLOCK` 主要维护 ABI 可见状态，不能解释为完整的换页锁定机制。这些边界不会削弱共享 frame 与延迟删除的主体语义，但限定了 System V IPC 的实现子集。

6.6 跨模块协作与功能总结

四种机制最终通过 TCB、fd table 和 `MemorySet` 汇合。表 6-8 总结了本章的关键设计成果及其依赖关系。

表 6-8 IPC/信号机制的跨模块协作

机制	依赖的内核边界	已形成的能力
信号	TCB 的线程组/可中断状态、trap context、scheduler wakeup	异步产生、线程选择、handler frame、默认动作与 sigreturn 完整闭环
管道	FdTable/FileOp 引用生命周期、blocked/ready 调度、poll waiters	阻塞与非阻塞字节流、EOF/EPIPE/SIGPIPE、poll/epoll 通知
futex	MemorySet backing 身份、哈希等待队列、timer 与 signal	wait/wake/bitset/requeue、超时和 signal 的 single-winner 竞争处理
System V shm	frame 所有权、VMA/PTE 建图、全局段表	shmget/shmat/shmdt/shmctl、共享物理页、失败回滚与延迟删除

综上, RespOS 的 IPC 正确性建立在同一组工程原则上: 先定义对象身份, 再登记可追踪的 waiter; 所有竞争事件只允许一个提交者; 跨模块状态修改采用 prepare/commit; 对象释放以最后引用、最后 attach 或安全调度点为边界。信号提供异步控制面, 管道提供流式数据面, futex 提供低开销同步, 共享内存提供直接数据共享, 四者共同构成了面向真实用户态运行库和工具链的进程协作基础。

7. 时钟模块

本章回答: RespOS 如何从两个架构的硬件计数器获得统一时间?时钟中断如何驱动调度和超时唤醒?用户态的 `clock`、`sleep`、`interval timer`、`POSIX timer` 和 `timerfd` 如何共享同一套时间基础?

时钟模块是内核中连接硬件、调度器和用户态时间接口的基础设施。它一方面读取架构相关的硬件计数器并设置下一次 `timer interrupt`, 另一方面把“经过了多少时间”转换为不同用途的时间: 用户可见的运行时间、超时和 `deadline` 使用的单调时间, 以及 CPU 时间记账使用的时间。`nanosleep`、`futex`、`interval timer`、`POSIX timer` 和 `timerfd` 都建立在这些统一的时间接口之上。

RespOS 将“读取时间”和“等待事件”分开处理: 读取时间只查询计数器并完成单位转换, 等待事件则登记 `deadline`、阻塞当前任务, 在时钟安全点检查到期对象并唤醒任务或投递信号。这种分工使不同用户态接口可以共享计时基础, 同时保持各自的可观察语义。

7.1 时间模型与核心数据

7.1.1 三种时间尺度

内核不会把所有时间需求都直接绑定到同一个频率。架构层分别提供三种逻辑时钟频率:

时间尺度	使用场景	设计目的
hardware clock	设置硬件中断、 <code>timeout</code> 和 <code>deadline</code>	反映底层计时器的真实运行尺度
user clock	<code>clock_gettime</code> 、文件时间和用户可见运行时间	提供稳定的用户态时间单位和精度
accounting clock	<code>times</code> 、 <code>getrusage</code> 等 CPU 时间记账	为资源统计保留独立的计时口径

表 7-1 RespOS 的时间尺度

在当前架构实现中, `get_time()` 读取硬件计数器, `get_time_ms/us()` 按 `user clock` 转换, `get_timeout_ms/us()` 按 `hardware clock` 转换, `get_accounting_ms/us()` 按 `accounting clock` 转换。这样, 超时判断不会因为用户可见时间的展示策略变化而改变, CPU 记账也不必与 `wall-clock` 语义混在一起。

7.1.2 `TimeSpec` 与 `deadline`

用户态时间接口统一使用 `TimeSpec` 表示秒和纳秒:

```
#[repr(C)]
pub struct TimeSpec {
    pub sec: isize,
    pub nsec: isize,
}

impl TimeSpec {
    pub fn is_valid_duration(&self) -> bool {
        self.sec >= 0 && self.nsec >= 0 && self.nsec < 1_000_000_000
    }

    pub fn checked_duration_us(&self) -> Option<usize> {
        if !self.is_valid_duration() {
```

```

        return None;
    }
    (self.sec as usize)
        .checked_mul(1_000_000)
        .and_then(|us| us.checked_add((self.nsec as usize).div_ceil(1000)))
    }
}

```

代码片段 7-1 时间值校验与单位转换

`TimeSpec` 的校验不仅检查秒和纳秒的范围, 还在转换时使用 `checked arithmetic`, 防止用户提供的超大时间值在内核中发生整数溢出。相对等待会被转换为 `now + duration` 的绝对 `deadline`; 绝对等待则直接使用目标时刻。之后各类定时器只需要比较当前时钟和 `deadline`, 而不必重复解释用户态结构体。

7.2 时钟源与时钟中断

7.2.1 RISC-V 与 LoongArch 的时钟源

公共内核只依赖架构层导出的 `get_time`、频率转换和 `set_next_ti_trigger` 接口, 具体硬件操作收敛在两个架构目录中。

RISC-V64。 RISC-V 通过 `time` 计数器读取当前 `tick`, 并通过 SBI `set_timer` 设置下一次 supervisor timer interrupt。`set_next_ti_trigger` 按硬件频率和固定 `tick` 周期计算下一次触发点, 当前系统每秒设置 100 次基础时钟检查。

LoongArch64。 LoongArch 通过 `rdtime.d` 读取稳定计数器, 并使用架构相关的 `timer CSR` 和 `set_timer` 设置下一次触发。启动阶段会通过 `CPUCFG` 读取并记录平台计时频率, 用于诊断; 运行时使用 `board` 配置提供的 `hardware`、`user` 和 `accounting clock frequency`, 避免公共代码依赖具体平台寄存器。

LoongArch 的时钟频率初始化必须在清零 BSS 之后执行, 因为初始化结果需要写入已经准备好的静态数据区域。随后两个架构都经过统一的 `rust_main_high` 启动路径: 初始化 `trap`、内存和任务系统, 开启 `timer interrupt`, 设置第一次触发时间, 最后进入调度器。

7.2.2 Timer trap 的职责

一次 `timer trap` 主要完成三件事: 重新编程下一次硬件触发、在安全点处理到期对象、请求当前任务被调度。其逻辑可以概括为图 7-1:

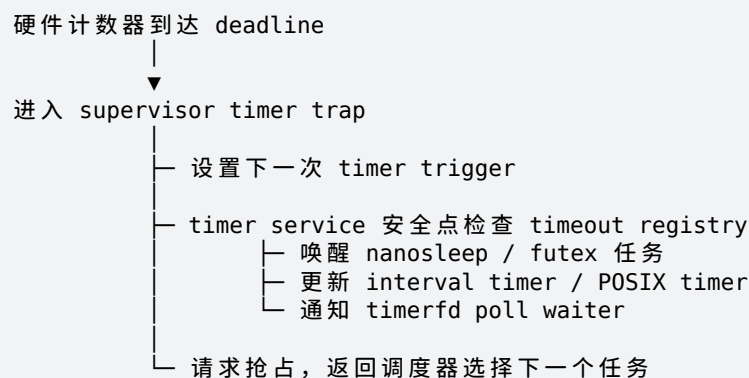


图 7-1 时钟中断到超时处理的调用链

在 RISC-V SMP 模式下, 全局 timeout 工作由 timer service hart 负责。用户态进入 timer trap 时可以直接处理; 内核态 timer trap 只有在该 hart 处于 per-CPU idle context、没有持有任务相关高层锁时才执行全局扫描, 其余 CPU 只重新设置本地 tick。这样既保留了多核下每个 CPU 的本地时钟, 又避免 timer 中断在任意 syscall 临界区重入 task、signal 或 timer registry。

LoongArch 当前在 timer trap 中直接完成统一的到期检查和抢占。无论采用哪种架构路径, timer trap 都只负责把硬件事件转换为内核时间服务; 具体的 sleep、signal 和 fd 语义仍由各自的模块维护。

7.3 超时管理与任务唤醒

7.3.1 deadline registry

RespOS 当前没有把所有定时器抽象成一个统一的时间轮或单一堆结构, 而是根据使用者的语义分别维护 deadline registry: nanosleep 等等待项按时钟类别和 deadline 组织, POSIX timer 使用全局 timer 表, 进程 interval timer 保存在任务控制块中, timerfd 则保留自己的周期和累计到期次数。这种设计让每种对象可以直接维护自身状态, 同时由 check_all_task_timers() 在统一安全点触发检查。

```
pub fn check_all_task_timers() {
    crate::task::check_futex_timeouts();
    check_nanosleep_timeouts();
    check_timerfd_expirations();
    crate::task::check_active_itimers();
    check_posix_timers();
}
```

代码片段 7-2 统一的到期检查入口

这个入口体现了时钟模块与其他子系统的边界: 时钟模块负责提供检查时机和当前时间, futex、task、signal 和 timerfd 模块负责解释到期事件。检查函数通常先在 registry 内确定已经到期的对象, 再在释放 registry 锁后执行唤醒或信号投递, 避免在持有底层容器锁时进入调度器或信号系统。

7.3.2 nanosleep 与阻塞唤醒

nanosleep 的实现首先校验用户提供的 TimeSpec, 使用 timeout clock 计算相对 deadline, 然后将当前任务登记到 nanosleep wait registry 并进入可中断阻塞。任务被唤醒后会重新检查三种情况: deadline 是否已经到达、是否收到信号、是否仍需继续等待。

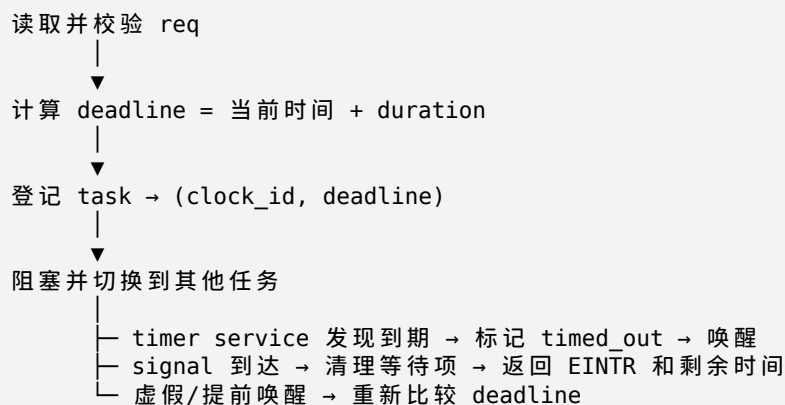


图 7-2 nanosleep 的登记、唤醒与返回流程

相对 `nanosleep` 使用 `timeout clock`, `clock_nanosleep` 还可以使用 `CLOCK_REALTIME`、`CLOCK_MONOTONIC` 和 `CLOCK_BOOTTIME` 的绝对 `deadline`。被信号打断时, 内核根据开始时间和请求时长计算剩余时间, 并在用户 `copyout` 成功后返回, 从而保证等待状态不会因为失败的用户地址而提前提交。

7.3.3 interval timer 与 POSIX timer

任务控制块为三类 `interval timer` 保存 `deadline`、`interval` 和对应 `signal`。timer 到期后, 内核使用原子比较交换更新下一次 `deadline`, 再向任务投递信号; 一次性 timer 清零 `deadline`, 周期 timer 则根据 `interval` 重新装载。全局 `ACTIVE_ITIMER_TASKS` 只记录拥有活动 timer 的任务, timer service 只扫描这些任务, 避免每次 tick 遍历所有进程。

POSIX timer 使用独立的全局表保存 timer id、owner、clock、signal、deadline 和 interval。timer_create 先准备对象并将 id copyout 到用户地址, 成功后才把对象发布到全局表; timer_settime 先读取并校验新值, 在 old value copyout 成功后再提交 deadline。进程组退出时, 按照 owner tgid 一次性移除该组创建的 timer, 避免 timer 残留后向已经复用的任务标识投递信号。

7.4 用户态时间接口

7.4.1 clock_gettime 与系统时间

用户态可以通过 `clock_gettime` 读取不同语义的时钟:

时钟	RespOS 中的含义	典型用途
<code>CLOCK_REALTIME</code>	monotonic time 加可调整的 realtime offset	日历时间、文件时间、绝对时间
<code>CLOCK_MONOTONIC</code>	持续增长的单调时间	timeout、相对等待和 deadline
<code>CLOCK_MONOTONIC_RAW</code>	原始单调时间口径	需要避免 realtime 调整的计时
<code>CLOCK_BOOTTIME</code>	当前运行期间持续增长的时间口径	睡眠与系统运行时间相关接口
<code>CLOCK_REALTIME_COARSE</code> / <code>MONOTONIC_COARSE</code>	对应时间量化到毫秒	低开销的粗粒度查询

表 7-2 用户可见时钟接口

`CLOCK_REALTIME` 的调整通过独立的 offset 完成, 不修改 monotonic 基准。因此, 设置系统时间不会让已经登记的 monotonic timeout 倒退或突然提前; 使用 realtime 的绝对 timer 则按照 realtime 语义计算当前 deadline。粗粒度时钟通过量化结果减少高精度转换开销。

7.4.2 timerfd: 将时间转换为文件事件

timerfd 将定时器包装成 FileOp, 使定时器可以与 poll/epoll 和普通文件描述符一起等待。每个 timerfd 保存 clock id、interval、deadline、已消费的 expiration count 和 poll waiter:

```
#[derive(Clone, Copy, Default)]
struct TimerFdState {
    interval_ms: usize,
    deadline_ms: usize,
    consumed: u64,
}
```

代码片段 7-3 timerfd 的核心状态

读取 `timerfd` 时，内核根据当前时钟计算已经发生的到期次数，并返回一个 `u64` 计数；周期 `timer` 即使用户态没有及时读取，也不会为每次到期创建独立事件，而是把多次到期合并为累计次数。非阻塞读取在没有 `pending expiration` 时返回 `EAGAIN`，阻塞读取则通过统一的 `poll waiter` 等待下一次通知。

`timerfd` 本身仍以文件描述符参与生命周期管理，关闭最后一个引用时由 `Arc` 回收对象。全局 `timerfd registry` 只保存弱引用，用于在 `timer service` 检查到期状态后通知可读等待者，不会因为 `registry` 而延长已经关闭的 `timerfd` 生命周期。

7.5 与其他模块的协作

时钟模块通过统一的当前时间和到期检查入口服务多个内核子系统：

协作模块	时钟模块提供的能力	产生的用户态效果
task / scheduler	timer interrupt、抢占时机、任务 timeout 唤醒	sleep、阻塞任务按 deadline 恢复执行
futex	timeout clock 和到期检查	FUTEX_WAIT 的超时返回
signal	interval/POSIX timer 到期事件	SIGALRM 等信号投递
FS / poll	timerfd 的可读通知	timerfd 与 poll/epoll 统一等待
syscall	clock、sleep、timer 的 ABI 转换	用户态读取时间和设置定时器
net	单调时间戳	TCP/UDP 协议栈的超时与时间推进

表 7-3 时钟模块与其他子系统的协作关系

时钟模块的设计成果可以概括为：架构相关的计数器和中断控制被封装在 `HAL` 层；公共内核获得统一的毫秒/微秒时间接口；不同类型的等待对象通过 `deadline registry` 接入同一套安全点；`sleep`、`futex`、`signal`、`timerfd` 和网络协议栈分别保留自己的状态机；用户态则可以通过 `clock`、阻塞等待、信号和文件事件四种方式使用时间能力。

这种设计把硬件时钟的差异限制在架构目录内，同时把时间服务自然地融入调度器、信号系统和文件描述符模型，既满足 `Linux` 风格用户态程序的时间接口需求，也为双架构和多核运行保留了清晰的扩展边界。

8. 网络模块

本章回答：RespOS 如何将 smoltcp 协议栈接入 Linux socket ABI？TCP、UDP 和 UNIX 域 socket 的生命周期如何管理？并发 `listen/accept` 为什么需要专门的 listener 池？

RespOS 采用“内核管理 ABI、对象生命周期和阻塞语义，smoltcp 管理 TCP/IP 协议状态机”的分层设计。这一边界使项目无需重新实现 TCP 重传、滑动窗口和报文解析，而能把工程重点放在用户态真正可见的部分：`fd` 身份、`bind/connect/listen/accept`、阻塞与信号中断、backlog 并发容量、`poll/epoll` 就绪性以及 `close` 时的句柄回收。

当前数据面明确聚焦于本地回环通信。内核已实现 IPv4 loopback 上的 TCP/UDP，并为 `AF_INET6` 的 `::/::1` 提供回环 ABI 兼容，同时实现基于内存队列的 UNIX 域 socket。源码中没有将 `virtio-net` 设备接入 `Interface`，因此本章不将真实以太网收发或物理网卡列为已完成能力。

8.1 分层架构与责任边界

RespOS 从 `syscall` 到数据包共分为五层。如图 8-1 所示，`syscall/net.rs` 负责 Linux ABI 结构解析和用户拷贝，`Socket` 负责 `fd/VFS` 统一抽象，`TcpSocket/UdpSocket` 管理内核可见状态，`smoltcp` 管理协议细节，`LoopbackDev` 则完成纯软件收发。



图 8-1 RespOS 网络模块分层调用链

表 8-1 进一步明确各层拥有的状态，以及不应越界处理的内容。

表 8-1 网络各层的所有者与职责

层次	主要对象	负责的状态	不在该层处理的内容
syscall ABI	SockAddrIn/SockAddrIn6/ SockAddrUn、MsgHdr、MMsgHdr	参数长度、字节序、用户指针、 fd 分配与 errno	TCP 状态转换、句柄回收
fd/VFS 抽象	Socket : FileOp	domain/kind、非阻塞状态、统 — read/write/readiness	TCP 报文和 UDP 队列算法
传输层适配	TcpSocket、UdpSocket、 ListenTable	连接粗粒度状态、本地/对端 地址、backlog、阻塞等待	拥塞控制、重传算法
协议栈	smoltcp SocketSet、TCP/UDP socket	TCP/IP 协议状态机、收发缓 冲、报文处理	Linux fd 和进程语义
设备	LoopbackDev	待接收数据包队列和可复用 缓冲池	真实 NIC 中断、DMA 和 virtqueue

这样分层后，更换协议库或增加真实网卡时，应优先保持 `Socket : FileOp` 和 `syscall ABI` 层不变；而修改 `accept` 并发性时，则必须同时审计 `ListenTable`、`smoltcp handle` 和 `fd` 的唯一所有权。

8.2 Socket 对象与 fd 生命周期

RespOS 通过让 `Socket` 实现 `FileOp` 将套接字纳入统一 `fd` 模型。这使 `read/write/close/fcntl/ppoll/epoll` 无需在文件和 `socket` 之间维护两套描述符表；对用户态而言，`socket fd` 与普通 `fd` 遵循相同的分配、复制和关闭边界，但通过 `ESPIPE` 明确拒绝 `seek`。

代码片段 8-1 用户可见 `socket` 的协议分派

```
// os/src/net/socket.rs (节选)
enum SocketInner {
    Tcp(TcpSocket),
    Udp(UdpSocket),
    Unix(UnixSocket),
}

pub struct Socket {
    pub domain: SocketDomain,
    pub kind: SocketKind,
    inner: SocketInner,
    nonblock: AtomicBool,
    cloexec: AtomicBool,
    send_buf_size: AtomicU64,
    recv_buf_size: AtomicU64,
}

impl FileOp for Socket {
    fn read<'a>(&'a self, buf: &'a mut [u8]) -> SysResult<usize> { /* 按协议分派 */ }
    fn write<'a>(&'a self, buf: &'a [u8]) -> SysResult<usize> { /* 按协议分派 */ }
    fn read_ready(&self) -> bool { self.tcp_poll(true) }
    fn write_ready(&self) -> bool { self.tcp_poll(false) }
}
```

代码片段 8-1 中的 `Socket` 是 `open-file description`: `dup` 或 `fork` 后的 `fd` 可以共享同一 `socket` 对象和连接状态。`SOCK_NONBLOCK` 在创建时传入对象，后续 `fcntl` 也经 `FileOp::set_status_flags` 更新相同状态。`SOCK_CLOEXEC` 进入 `fd` 旗标语义，使 `exec` 能经统一 `fd table` 路径关闭描述符。

socket 对象的创建与用户 copyout 还遵循失败原子性。例如 socketpair 会先检查用户结果数组，再依次分配两个 fd；第二个 fd 分配或最终 copyout 失败时，已分配的 fd 会被回收。accept4 创建新 socket 并分配 fd 后，若写回对端地址失败，也会关闭新 fd，不向进程留下不可达的连接对象。

8.3 回环设备与协议轮询

RespOS 将回环通信实现为符合 smoltcp Device trait 的纯软件设备。LoopbackDev 不经过 virtqueue 或 MMIO：发送 token 从缓冲池取出 Vec<u8>，smoltcp 填充报文后将其推入接收队列；下一次 interface poll 再通过接收 token 取出同一缓冲。接收 token 销毁时将缓冲归还池中，池最多保留 64 个缓冲，避免高频回环报文不断重新分配堆对象。

```
smoltcp TxToken::consume
  → 从 pool 取缓冲/按需分配
  → 填充 IP + TCP/UDP 报文
  → 进入 LoopbackDev.queue
  → Interface::poll()
  → RxToken::consume
  → 投递到对端 smoltcp socket
  → RxToken drop, 缓冲返回 pool
```

图 8-2 回环设备的纯软件数据路径

图 8-2 中的全局 SocketSet、loopback Interface、LoopbackDev 和 ListenTable 均由锁保护。poll_interfaces() 以当前内核时间驱动 Interface::poll()，使 smoltcp 处理新报文、TCP 状态转换和缓冲投递；随后立即把已完成握手的 listener handle 转入 accept queue 并补充新 listener。该 poll 入口由 TCP/UDP 阻塞循环、socket readiness 查询和 /proc/net/tcp 生成路径调用，因而当前协议栈是协作式驱动，而不是由独立 NIC 中断线程推进。

8.4 TCP 套接字与并发监听

RespOS 在 smoltcp TCP 状态机之上增加了面向 syscall 并发操作的粗粒度状态。CLOSED、CONNECTING、CONNECTED 和 LISTENING 表达用户可见生命周期，BUSY 则是临时 CAS 中间态，用于防止同一 socket 被两条路径同时执行不可并发的状态转换。

代码片段 8-2 TCP 粗粒度状态的 CAS 提交

```
// os/src/net/tcp.rs (节选)
fn update_state<F, T>(&self, current: u8, new: u8, f: F) -> Result<T, Errno>
where
    F: FnOnce() -> Result<T, Errno>,
{
    match self
        .state
        .compare_exchange(current, STATE_BUSY, Ordering::Acquire, Ordering::Acquire)
    {
        Ok(_) => {
            let res = f();
            if res.is_ok() {
                self.set_state(new);
            } else {
                self.set_state(current);
            }
            res
        }
        Err(_old) => Err(Errno::EISCONN),
    }
}
```

```
}
}
```

代码片段 8-2 的关键在于失败时恢复原状态。`bind`、`connect`、`listen`、`close` 都通过同一入口提交粗粒度转换，从而避免协议句柄已创建而用户状态仍留在另一阶段。

```
客户端: socket → [bind] → connect → CONNECTING
          ↓ poll
          CONNECTED → send/recv → shutdown/close

服务端: socket → [bind] → listen → LISTENING
          ↓
          accept → 新 CONNECTED socket
```

图 8-3 TCP 客户端与服务端生命周期

图 8-3 展示的状态转换由 `syscall` 操作触发，而协议握手的实际推进依赖下一节所述的协作式轮询。

8.4.1 阻塞、非阻塞与信号中断

TCP 的 `connect/accept/send/recv` 共用 `block_on` 模式：先调用 `poll_interfaces()` 推进协议栈，再尝试一次操作；如果返回 `EAGAIN`，非阻塞 `socket` 立即向用户返回，阻塞 `socket` 则将当前任务标记为可中断等待，登记约 1 ms 的短期 `timeout` 后切换任务，醒来再重试。

之所以不在 `syscall` 中持续忙轮询，是因为 `listener` 等待第一个连接时可能暂时没有其他 `ready` 任务；如果内核一直在 TCP 循环中占用 CPU，调度器 `idle` 路径无法推进 `sleep/timeout`，无关定时任务也会被饥饿。短期阻塞让出 CPU，同时保留轮询式协议栈的实现简洁性。

等待期间会在切换前后检查 `real timer`、`pending signal` 和 `interrupted` 状态，被信号中断时返回 `EINTR`。注册 `blocked` 与真正切换之间的信号窗口也会二次检查，避免发送方已标记中断、但尚找不到 `blocked queue` 条目时产生丢失唤醒。

`shutdown(SHUT_RD)` 记录接收半边关闭，后续读取返回 `EOF`；`SHUT_WR` 记录发送半边关闭并调用 `smoltcp close()` 推进 `FIN` 路径，后续写入返回 `EPIPE`。`Drop` 是所有权最终落点：普通连接从全局 `SocketSet` 移除 `handle`，监听 `socket` 还由 `unlisten` 回收 `listener` 池和 `accept queue` 中尚未交给用户的所有 `handle`。

8.4.2 backlog listener 池

RespOS 不把 `smoltcp` 的单个 `listening socket` 直接等同于 Linux `backlog`，因为 `smoltcp` 中一个 `listener` 完成握手后会变成该条已连接 `socket`。在用户态调用 `accept` 前，原端口就暂时失去可承接后续 `SYN` 的 `listener`；多个 `client` 在同一轮 `poll` 中到达时，后续连接可能收到 `reset`。

RespOS 因此将 `listen(backlog)` 实现为受限 `listener` 池。`backlog` 被限制到 1~128，每个端口的 `ListenTableEntry` 同时维护“仍在监听/握手中”的 `listen_handles` 和“已连接、等待 `accept`”的 `accept_queue`。两个队列总长度不超过 `backlog`，既预留并发握手容量，又防止恶意大 `backlog` 无界占用每个 64 KiB 收发缓冲的 TCP `socket`。

代码片段 8-3 `listener` 池的容量不变量

```
// os/src/net/listen.rs (节选)
fn replenish_listeners(entry: &mut ListenTableEntry) {
```

```

while entry.listen_handles.len() + entry.accept_queue.len() < entry.queue_limit {
    let mut socket = SocketSetWrapper::new_tcp_socket();
    if socket.listen(entry.listen_endpoint).is_err() {
        return;
    }
    let handle = socket_set().lock().add(socket);
    entry.listen_handles.push_back(handle);
}
}

```

如代码片段 8-3 所示，池中每移出一个已连接 handle，就尽快补入新 listener。这一补充不等待用户态 accept 才发生：每次协议 poll 后，promote_ready_listeners 会把 Established handle 转入 accept queue，然后立即补位。

```

listen(backlog=N)
→ 预建 N 个 listener handle (N 限制为 1..128)
→ Interface::poll 完成一条握手
→ Established handle 从 listen_handles 移入 accept_queue
→ 立即补建新 listener
→ accept 取出 handle，所有权转入新 TcpSocket

```

图 8-4 backlog listener 池与 accept queue 的句柄转移

图 8-4 中的 handle 只是 SocketSet 索引，本身不拥有协议对象，因此每条移交路径都明确唯一所有者：监听期属于 ListenTableEntry，accept 成功后属于新 TcpSocket，unlisten 则显式 remove 两个队列中的所有剩余 handle，从而避免重复 remove 和未接受连接泄漏。

该设计来自对真实并发失败的修复。旧实现只有一个 listener，多个 client 并发连接时会随机 reset。2026-08-03 的 RV64 CAgent 回归中，修复后连续三轮不再出现 Connection failed to 127.0.0.1:8080，包括 network 在内的网络责任项稳定通过；当时整体 8/10 的剩余失败已由日志定位到文件系统路径，不被用来扩大网络结论。

8.5 UDP 与 UNIX 域 socket

8.5.1 UDP 数据报

UDP 不需要 TCP 的连接状态机，但仍然管理本地端点、默认对端、端口冲突、阻塞语义和数据报边界。UdpSocket::new 在全局 SocketSet 中创建一个带 256 个 metadata slot、64 KiB 接收字节区和 64 KiB 发送字节区的 smoltcpsocket。

bind 检查同地址/端口是否已被现有 UDP socket 占用；端口为 0 时，从 0xc000..=0xffff 循环取候选端口并扫描 SocketSet 验证冲突。因而，当前源码中没有独立 PortManager，端口所有权由存活的协议 socket 直接体现。

表 8-2 对比无连接与保存默认对端两种 UDP 用法。

表 8-2 UDP 两种用法的数据路径

用法	发送	接收	地址语义
无连接	sendto(buf, remote)	recvfrom(buf)	每个数据报单独指定/返回对端
已 connect	send(buf)	recv(buf)	socket 保存默认对端，接收时过滤非匹配端点

未显式 `bind` 的 `socket` 在首次发送或 `connect` 时自动绑定回环临时端口。`recvfrom` 先使用固定 1528 字节的内核临时缓冲接收, 再按用户缓冲长度复制并写回来源地址; 协议栈报告数据报超过该临时缓冲时, 当前路径将 `Truncated` 转换为 `EAGAIN`, 因此不能把它表述为任意长度数据报的完整接收。阻塞 UDP 会在 `EAGAIN` 时让出 CPU 并检查信号, 但当前使用 `cooperative yield`, 没有 TCP 路径的 1 ms `blocked-timeout` 协议。

8.5.2 UNIX 域 socket

RespOS 为本机 IPC 提供不经过 `smoltcp` 的 UNIX 域 `socket`, 因为 `socketpair` 和基于 `pathname` 的本地服务是 `libc`、`shell` 和多进程工具的常见依赖, 不需要 IP 报文开销。每个 `UnixSocket` 持有一个接收字节队列和对端队列引用, 单向队列容量上限为 64 KiB; `socketpair` 直接交叉连接两个对象的队列。

`pathname bind` 会通过 VFS 创建 `InodeType::Socket` 节点, 同时把路径 `key` 注册到内核 `UNIX_LISTENERS` 表; 抽象地址只使用内核 `key`, 不创建路径节点。`listen` 为 `key` 建立 pending queue, `connect` 创建与客户队列交叉引用的服务端 `UnixSocket` 并推入队列, `accept` 取出该对象。pending queue 上限为 128, 监听 `socket` 关闭时删除对应注册项。

当前 `SOCK_STREAM`、`SOCK_DGRAM` 和 `SOCK_SEQPACKET` 的 UNIX 域数据面都复用字节队列, 因而已提供本地传输和 `socketpair`, 但 `DGRAM/SEQPACKET` 还没有独立保留消息边界的队列语义。这是当前实现的精确边界, 因此不将其表述为完整 UNIX datagram/seqpacket 实现。

8.6 网络系统调用与就绪性

RespOS 的网络 `syscall` 层保持“解析 ABI、获取 `socket` 对象、调用领域方法、写回结果”的边界。表 8-3 展示已接入的主要接口及其实现重点。

表 8-3 主要 `socket` 系统调用

类别	系统调用	RespOS 实现重点
创建	<code>socket</code> 、 <code>socketpair</code>	<code>domain/type/protocol</code> 校验, <code>SOCK_NONBLOCK/CLOEXEC</code> , 多 fd 失败回滚
端点与连接	<code>bind</code> 、 <code>connect</code> 、 <code>listen</code> 、 <code>accept/accept4</code>	<code>sockaddr</code> 转换, 低端口权限, <code>backlog</code> 传递, 新连接 fd 所有权
数据	<code>sendto/recvfrom</code> 、 <code>sendmsg/recvmsg</code> 、 <code>sendmmsg/recvmmsg</code>	通过 MM 逐页 <code>copyin/copyout</code> , <code>iovec</code> 聚合/分散, 部分成功返回
查询与控制	<code>getsockname/getpeername</code> 、 <code>setsockopt/getsockopt</code> 、 <code>shutdown</code>	长度受限写回, <code>TCP_NODELAY</code> 、IP TTL 等有效选项, 半关闭语义
等待	<code>read/write</code> 、 <code>ppoll/pselect6</code> 、 <code>epoll</code>	通过 <code>FileOp::read_ready/write_ready</code> 统一扫描 <code>socket</code> 可读/可写状态

`sendmsg/recvmsg` 会先检查 `IOV_MAX=1024` 和累计长度溢出, 再经 MM 的用户页拷贝路径处理每个 `iovec`; `sendmmsg/recvmmsg` 按消息顺序处理, 中途失败时, 若前面已有成功项就返回已完成数量, 保留 Linux 多消息接口的部分进度语义。控制消息、out-of-band 数据和 error queue 尚未实现, 相关 `flag` 返回明确 `errno`, 而不伪造附加数据。

`socket readiness` 由 `Socket::tcp_poll` 先驱动一轮协议 `poll`, 再查询 TCP/UDP/UNIX 对象。TCP 已连接 `socket` 根据 `can_recv/can_send/may_recv/may_send` 计算可读可写, `listener` 根

据 `accept queue` 是否非空可读, UDP 根据数据报缓冲状态判断, UNIX `socket` 则检查字节队列或 `pending queue`。这使 `socket` 能被标准 `ppoll/pselect6/epoll` 扫描; 当前 `socket` 没有覆盖 `FileOp::register_poll_waiter` 建立事件推送, 因而统一等待层对这类 `fd` 使用协作式重试, 不将其宣称为完全事件驱动的 `socket` 唤醒。

`/proc/net/tcp` 也复用同一实时状态。 `procfs` 读取时先 `poll` 协议栈, 再同时收集 `ListenTable` 中的 `listener/未 accept 连接` 和 `SocketSet` 中的 TCP `socket`, 转换为 Linux 工具可识别的端点与状态表。这使 `ss` 等用户工具能观察内核网络状态, 无需一份独立的 `procfs` 镜像缓存。

8.6.1 回环 IPv6 兼容边界

RespOS 接受 `AF_INET6` TCP/UDP `socket`, 但当前只对 `::` 和 `::1` 建立回环兼容: `syscall` 层将两者分别归一为 IPv4 `0.0.0.0` 和 `127.0.0.1` 再进入现有回环路径, 写回 `sockaddr_in6` 时再还原为全零或 `::1`。其他 IPv6 地址返回 `EADDRNOTAVAIL`。

这一设计的目标是满足只需本地回环的 `libc` 和应用探测, 而不是宣称已建立 IPv6 接口地址、邻居发现和路由。虽然 `smoltcp` 依赖启用了 `proto-ipv6` 特性, 可编译的第三方能力不自动等于本项目已将其连接到设备、地址配置和 ABI 的完整路径。

8.7 与其他模块的协作

网络模块的可用性依赖内存、VFS、调度、时钟和 `procfs` 的共同语义。这些交互是 `socket` 在竞赛负载中能正确阻塞、被中断并随进程回收的前提, 具体边界见表 8-4。

表 8-4 网络模块的跨模块协作

协作模块	网络使用的能力	关键约束
MM	<code>sockaddr</code> 、 <code>iovec</code> 、数据缓冲和 <code>sockopt</code> 的 <code>copyin/copyout</code>	用户页可能跨页或 <code>lazy/COW</code> , 不在 <code>net syscall</code> 中直接解引用
VFS / <code>fd table</code>	Socket : <code>FileOp</code> 、 <code>fd</code> 分配、 <code>CLOEXEC</code> 、 <code>O_NONBLOCK</code>	<code>descriptor flag</code> 与共享 <code>socket</code> 状态必须维持既有分层
<code>task / signal</code>	可中断阻塞、 <code>yield</code> 、 <code>blocked registration</code> 、 <code>EINTR</code>	检查与入队之间不得丢失 <code>signal wakeup</code>
timer	TCP 阻塞轮询的短 <code>timeout</code> 、 <code>smoltcp poll timestamp</code>	不在 <code>socket syscall</code> 忙等导致全局 <code>timeout</code> 饥饿
<code>procfs</code>	<code>/proc/net/tcp</code> 的动态端点和状态	从实际 <code>listener/socket</code> 生成, 不维护会过期的镜像

8.8 功能与设计成果总结

RespOS 网络模块的项目贡献主要位于 `smoltcp` 之上和之下的工程连接: 向上建立 Linux `socket` ABI、`fd/VFS` 身份和可中断阻塞语义, 向下建立可复用缓冲的 loopback **Device**, 并在中间为 `smoltcp` 单 `listener` 建立符合 `backlog` 用户语义的受限句柄池。表 8-5 汇总这些成果及其当前边界。

表 8-5 网络模块成果与当前边界

方向	已实现的设计成果	证据或边界
TCP loopback	<code>connect/listen/accept/send/recv/shutdown</code> , CAS 状态机, 可中断阻塞	<code>net_loopback_smoke</code> 覆盖 <code>fork</code> 客户端与请求/回显; 真实外网不在当前数据面内
并发监听	<code>backlog</code> 限制为 1~128, <code>listener</code> 池、 <code>accept queue</code> 和 <code>poll</code> 后立即补位	2026-08-03 RV64 CAgent 连续三轮的并发 <code>connect</code> 失败消失; 不用 <code>client</code> 硬编码重试掩盖 <code>ECONNREFUSED</code>
UDP loopback	<code>bind/sendto/recvfrom/connect/send/recv</code> , 自动临时端口	<code>net_loopback_smoke</code> 覆盖数据报发送、内容和来源地址
UNIX 域	<code>pathname/抽象 key</code> 、 <code>socketpair</code> 、 <code>listen/connect/accept</code> 、64 KiB 内存通道	<code>DGRAM/SEQPACKET</code> 当前与 <code>stream</code> 共用字节队列, 不宣称完整消息边界
Linux ABI	<code>sockaddr v4/v6/unix</code> 、 <code>msg/mmsg</code> 、主要 <code>sockopt</code> 、 <code>fd readiness</code> 、 <code>/proc/net/tcp</code>	<code>ancillary data</code> 、 <code>OOB/error queue</code> 、 <code>raw socket</code> 未实现; <code>socket poll waiter</code> 仍为协作式重试
网络层与设备	IPv4 loopback 与 <code>AF_INET6 ::::1</code> 兼容入口	其他 IPv6 地址、路由、 <code>virtio-net</code> 与物理网卡数据路径尚未接入

综上, RespOS 没有将第三方协议库的可编译功能简单等同于项目成果, 而是以真实调用链和用户可见语义划定边界。当前网络模块已经是一条闭环的本地通信路径: 套接字能被 `fd` 拥有、被 `smoltcp` 驱动、在阻塞时与调度/信号协作、在并发握手时维持 `backlog` 容量, 并在 `close` 后把协议句柄归还唯一所有者。

9. 硬件抽象层与架构适配

本章回答：RespOS 如何在 RISC-V64 和 LoongArch64 上复用同一套内核逻辑？架构层如何统一页表、trap、任务切换、时钟和启动流程？

RespOS 的双架构支持采用“公共内核逻辑 + 架构目录特化”的设计。任务管理、内存管理、文件系统和系统调用等模块使用统一的公共代码；启动入口、寄存器访问、页表操作、异常返回、上下文切换和时钟触发等必须依赖硬件的部分，则分别位于 `os/src/arch/rv64/` 和 `os/src/arch/loongarch64/`。

当前架构层不是一套完全由 `trait` 组成的独立 HAL 库，而是由条件编译、模块重导出、同名函数和相同的数据结构约定共同形成的适配边界。`os/src/arch/mod.rs` 根据 `target_arch` 选择具体实现并重新导出，使公共代码可以调用 `read_mmu_token`、`write_mmu_token`、`sfence`、`set_next_ti_trigger` 等统一入口，而不直接散落 CSR、汇编指令和硬件寄存器假设。

9.1 架构适配的整体结构

9.1.1 一套公共调用链，两套底层实现

RespOS 的架构目录按照启动、内存、trap、task、timer 和 interrupt 等职责组织。两套目录提供相近的模块接口，但在内部使用各自架构的寄存器和汇编实现：

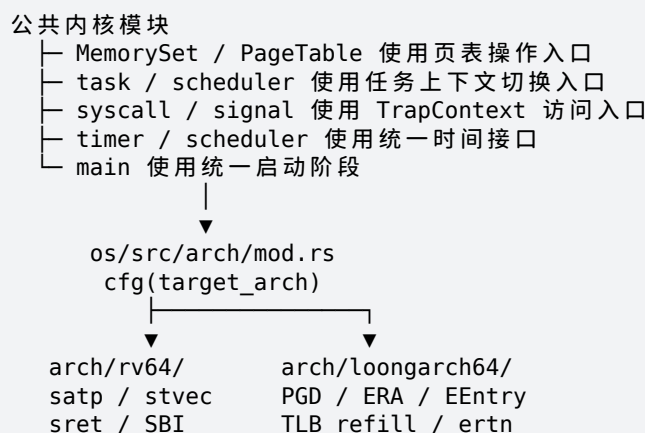


图 9-1 RespOS 双架构适配结构

这种结构的关键不是让两个架构的代码完全相同，而是让公共模块只依赖稳定的语义接口。例如，公共内存管理只需要“写入当前页表根并刷新地址转换缓存”，不需要知道 RV64 使用 `satp`、`sfence.vma`，还是 LoongArch 使用 `PGDL/PGDH` 和 `invtlb`。架构层负责把同一语义翻译成对应指令序列。

9.1.2 双架构差异总览

适配维度	RISC-V64	LoongArch64	公共抽象
内核入口	OpenSBI 进入 S-mode, 传入 hart id 和 FDT opaque	QEMU/固件进入低地址直映入口	<code>enter_main</code> → <code>rust_main_high</code>
当前计数器	<code>time</code> CSR/计数器	<code>rdtime.d</code>	<code>get_time</code> 、频率转换和下一次触发
地址转换根	<code>satp</code>	PGDL/PGDH	<code>read_mmu_token</code> 、 <code>write_mmu_token</code>
页表遍历	硬件 Sv39 遍历	TLB <code>refill</code> 汇编使用 <code>lddir/ldpte</code>	<code>map/unmap/query/protect</code>
用户 trap 状态	<code>stvec</code> 、 <code>sstatus</code> 、 <code>sepc</code> 、 <code>scause</code> 、 <code>stval</code>	EEntry、PRMD、ERA、ESTAT、BADV	TrapContext 与公共 trap handler
异常返回	<code>sret</code>	<code>ertn</code>	<code>restore</code> 汇编后的用户态返回
任务页表切换	写 <code>satp</code> 后 <code>sfence.vma</code>	同时写 PGDL/PGDH 后刷新 TLB	TaskContext 保存地址空间 token
外部设备形态	QEMU virt 的 MMIO 设备	QEMU LoongArch 的 PCI 设备	<code>drivers</code> 层统一设备操作

表 9-1 RV64 与 LA64 的架构差异及统一接口

9.2 地址空间与页表适配

9.2.1 共享高半区内核模型

两个架构最终采用相同的高半区内核地址模型。公共内核代码通过统一的 `KERNEL_BASE` 访问内核映像、内核堆和内核设备映射；创建用户地址空间时，架构层提供 `PageTable::from_kernel()`，复制内核根页表中属于内核高半区的部分。

这种设计让 `MemorySet` 可以在两种架构上复用相同的 VMA、页帧和映射逻辑。用户地址空间的差异主要集中在页表项编码、地址转换寄存器和 TLB 刷新，而不会扩散到 COW、lazy allocation、file-backed mapping 等上层机制。

9.2.2 RISC-V Sv39 页表

RISC-V 页表采用 Sv39 三级结构。`PageTable` 保存根页表物理页号和由 `FrameTracker` 管理的页表页帧；映射时按 VPN 的三级索引逐级创建中间页表，叶页表项保存物理页号和读写执行权限。`token()` 将根页表物理页号编码为 `satp`，任务切换时由汇编保存和恢复该 token。

RISC-V 还为内核 direct map 提供了大页映射入口。启动阶段的 early page table 先建立能够覆盖 QEMU 最大可达内存窗口的映射，之后由 FDT 识别实际 RAM 末址，并由正式 frame allocator 严格按实际内存上限分配。这样既能在早期读取固件提供的设备树，又不会把预留的最大窗口误认为真实物理内存。

9.2.3 LoongArch 页表与 TLB refill

LoongArch 的页表同样采用三级、4 KiB 页粒度的结构，但硬件地址转换通过 TLB 完成。发生 TLB miss 时，`tlb_refill.S` 使用 PGD 根地址和 `lddir`、`ldpte` 指令读取多级页表项，装载偶数页和奇数页的 TLBLo 后执行 `tlbfill`；如果目录项无效，则构造无效 TLB 项并返回异常路径。

LoongArch 的 `read_mmu_token` 返回 PGDL 根地址, `write_mmu_token` 同时更新 PGDL 和 PGDH, 并设置 ASID 和页表根同步状态。任务切换汇编也保存一个根 token, 恢复时写入两个根页表寄存器并刷新 TLB。公共代码因此只看到“切换地址空间并使地址转换状态生效”, 不需要处理两个根寄存器的细节。

9.2.4 页表操作的统一语义

两套 PageTable 都提供创建、映射、解除映射、查询、修改权限和替换页表项等操作。公共 MemorySet 依靠这些操作实现:



图 9-2 公共内存管理与架构页表实现的关系

页表页帧由页表对象持有并自动追踪。进程退出或地址空间替换时, 页表页帧不会立即在任意执行上下文中复用, 而是经过有限的 quarantine 队列延迟回收, 避免硬件仍可能观察旧页表根时发生页表页帧复用。这一机制在两套架构中保持相同语义, 只在页表根和刷新指令上有所不同。

9.3 Trap 与特权级切换

9.3.1 统一的 trap 三阶段

两种架构的 trap 入口虽然使用不同的 CSR 和汇编语法, 但都遵循相同的三阶段流程:



图 9-3 双架构 trap 处理的公共流程

RISC-V 从 `stvec` 进入 `trap.S`, 硬件提供 `sepc`、`scause` 和 `stval`; LoongArch 从 `EEntry` 进入对应入口, 硬件提供 `ERA`、`ESTAT` 和 `BADV`。两条路径随后都把通用寄存器和返回地址组织成 `TrapContext`, 交给公共 `syscall`、`page fault`、`timer` 和 `signal` 逻辑处理。

9.3.2 TrapContext 的统一字段与架构差异

`TrapContext` 表示一次用户态执行被内核接管时需要保存的完整状态。两个架构都保存 32 个通用寄存器、用户返回 PC、用户栈指针以及返回特权级所需的状态, 但字段形式不同:

内容	RISC-V64	LoongArch64
通用寄存器	x[32], ABI 参数使用 a0-a7	x[32], ABI 参数使用 r4-r11
返回地址	sepc	era
前一特权级	sstatus.SPP	prmd.PPLV
返回中断状态	sstatus.SPIE/SIE	prmd.PIE
浮点状态	f[32]、fcsr	当前 TrapContext 不单独保存浮点寄存器
公共访问	get_a0/set_a0/get_sp/set_sepc 等	提供同名语义访问方法, 内部映射到不同寄存器编号

表 9-2 两种架构的 TrapContext 对照

公共 `syscall` 代码通过 `get_a0`、`set_a0`、`get_sepc` 等语义方法访问上下文，不直接读取 `x[10]` 或 `x[4]`。例如，RISC-V 的 `syscall` 返回值写入 `x[10]`，LoongArch 则写入 `x[4]`，但公共代码只需要调用 `set_a0`。这使 Linux ABI 参数处理和系统调用分发逻辑能够跨架构复用。

9.3.3 返回用户态的安全顺序

RISC-V 的用户态返回路径包含一个必须保持的安全顺序：恢复过程中先清除 SIE，保持 `stvec` 指向内核 trap 入口，完成通用寄存器、浮点寄存器和 `sscratch` 等状态恢复后，才切换到用户 trap vector，最后执行 `sret`。用户态中断使能由 `sstatus.SPIE` 在 `sret` 时恢复。

如果提前恢复用户态 trap vector 或提前打开 SIE，timer 可能在 `sscratch` 和寄存器恢复尚未完成时重入用户 trap 入口，破坏 trap 保存约定。该顺序同时适用于普通 `syscall` 返回、`exec` 初始上下文和 signal `sigreturn`，因此被放在架构汇编层统一保证。

LoongArch 返回路径则恢复 PRMD、ERA 和通用寄存器，最后通过 `ertn` 返回。两者的返回指令不同，但共同遵守“完成内核上下文恢复后再恢复用户态特权状态”的原则。

9.4 任务上下文与调度切换

9.4.1 TaskContext 与 TrapContext 的分工

RespOS 将用户态异常上下文和内核态调度上下文分开。`TrapContext` 保存用户程序被中断时的完整执行状态，供 `syscall`、信号和 page fault 处理使用；`TaskContext` 只保存一次内核任务切换所需的调用约定寄存器、内核栈位置和地址空间 token，供 scheduler 的 `__switch` 使用。

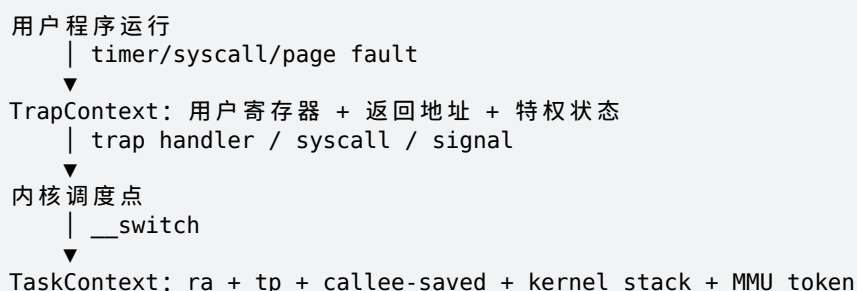


图 9-4 TrapContext 与 TaskContext 的职责分工

这种分工避免每次内核调度都复制完整用户寄存器集合。任务被阻塞或主动让出 CPU 时，架构汇编只保存当前内核执行点能够继续运行所需的最小上下文；任务重新获得 CPU 后，从自己的内核栈恢复并继续执行。

9.4.2 两套 switch.S 的共同布局

RISC-V 和 LoongArch 的 `switch.S` 使用相同的抽象参数：下一个任务的内核栈指针和当前任务控制块中的栈指针位置。两套汇编都保存 `ra`、`tp`、`callee-saved` 寄存器和当前地址空间 token，然后恢复下一个任务的对应内容，切换栈并返回。

差异集中在寄存器名称、加载存储指令和地址转换刷新指令：RISC-V 写入 `satp` 后执行 `sfence.vma`；LoongArch 写入 `PGDL/PGDH` 后执行数据屏障、TLB invalidate 和指令屏障。调度器不需要知道这些差异，因此任务调度和阻塞唤醒逻辑保持在公共 `task` 模块中。

9.5 启动流程适配

9.5.1 公共启动阶段

两个架构最终都会进入 `rust_main_high`，公共启动顺序为：

架构入口

- ├─ 清零 BSS，建立早期栈和必要的地址映射
- ├─ 进入高半区公共 Rust 入口
- ├─ 初始化 trap
- ├─ 初始化内存管理
- ├─ 初始化网络和 initproc
- ├─ 初始化多核 idle task (RV64)
- ├─ 开启 timer interrupt 并设置第一次触发
- └─ 进入 task::run_tasks

图 9-5 双架构公共启动阶段

公共启动顺序保证了依赖关系: trap 必须在处理异常前初始化, 内存必须在创建任务和网络对象前可用, initproc 入队后才开始调度, timer interrupt 则在调度器准备好之后开启。

9.5.2 RISC-V 启动路径

RISC-V `_start` 从 OpenSBI 进入 S-mode, 获得 hart id 和 FDT opaque 参数。汇编入口根据 hart id 选择独立 early stack, 写入 boot page table 的根到 `satp`, 执行 `sfence.vma` 后跳转到 `enter_main`。Rust 入口随后完成 BSS 清零、读取 FDT 中的实际内存范围, 并进入公共启动阶段。

RISC-V 的启动页表预先覆盖 QEMU virt 的最大可达内存窗口, 以保证 boot hart 能访问 OpenSBI 放置在 RAM 顶部的 FDT; 正式内存管理仍依据 FDT 的实际 `reg` 范围建立 frame allocator 和 kernel direct map。这体现了“早期可达窗口”和“真实资源上限”的明确分工。

9.5.3 LoongArch 启动路径

LoongArch `_start` 进入时仍处于低地址直接映射环境。入口先保留当前执行段所需的 DMW 配置, 再开启低地址 DMW, 使早期代码可以访问低物理地址; 随后 `enter_main` 跳转到 Rust `rust_main`。在公共启动前, LoongArch 建立覆盖早期内核镜像和内核堆的临时高地址页表, 开启分页并跳转到共享高半区内核入口。

这一过渡解决了两个地址模型之间的切换问题: 启动入口可以依赖固件提供的低地址直映, 公共内核则可以从一开始使用与 RISC-V 相同的高半区地址约定。进入正式页表后, `LOW_DIRECT_MAP_ACTIVE` 等架构状态用于记录早期直映是否仍在使用的, 避免公共代码误把启动阶段映射当成最终地址空间模型。

9.6 功能与设计总结

RespOS 的硬件抽象与架构适配可以归纳为表 9-3:

设计成果	实现方式
公共内核跨架构复用	<code>arch/mod.rs</code> 使用条件编译选择架构实现，并通过同名入口向公共模块提供统一语义
双架构地址空间	两个架构共享高半区内核模型，分别封装 <code>satp</code> 或 <code>PGDL/PGDH</code> 、页表遍历和 TLB 刷新
统一 trap 处理	架构汇编保存不同硬件上下文，公共 Rust handler 复用 <code>syscall</code> 、 <code>page fault</code> 、 <code>timer</code> 和 <code>signal</code> 分发
统一任务切换	<code>TaskContext</code> 采用相同抽象布局， <code>switch.S</code> 仅特化寄存器保存、页表根切换和地址转换刷新
双架构启动	RV64 从 OpenSBI/FDT 进入，LA64 从低地址直映过渡到高半区，最终进入同一公共初始化顺序
架构安全约束	在 <code>arch</code> 层保证 <code>trap return</code> 顺序、页表根切换、TLB 刷新和早期映射过渡的正确性

表 9-3 硬件抽象层与架构适配的设计成果

总体而言，RespOS 没有试图抹平 RISC-V 和 LoongArch 的硬件差异，而是把差异收敛到入口、寄存器、页表、trap 汇编和任务切换等少数边界。公共内核因此可以围绕统一的任务、地址空间和系统调用语义持续演进；架构层则负责将这些语义准确落实到两套不同的指令集和特权体系上。

10. 设备驱动

本章回答：设备驱动如何抽象？virtio 块设备和网卡如何接入内核？

10.1 概述

RespOS 的驱动模型通过统一的 **Device trait** 和 **Disk trait** 抽象块设备，通过 virtio 框架支持 virtio-blk 和 virtio-net 两种虚拟设备。设备驱动层位于 HAL 之上——HAL 提供 MMIO 地址映射和中断注册能力，驱动层负责设备初始化和数据传输协议。

【建议检查的代码】os/src/drivers/mod.rs, os/src/drivers/device.rs

10.2 块设备驱动

virtio-blk 设备初始化。virtio-blk 是 QEMU 提供的高性能虚拟块设备。初始化流程为：通过 MMIO 地址探测 virtio 设备 → 读取设备 ID 确认是块设备 → 协商 features（如 VIRTIO_BLK_F_R0、VIRTIO_BLK_F_FLUSH 等）→ 分配一个 virtqueue → 完成驱动初始化。

块读写接口。**Disk trait** 提供 `read_block(block_id, buf)` 和 `write_block(block_id, buf)` 两个核心操作。每个请求构造一个 virtio-blk request (type + sector + data)，填入 virtqueue descriptor，通知设备后轮询等待响应。数据直接从 descriptor 对应的物理内存区域传输，无需额外拷贝。

与文件系统的对接。ext4 后端的 **BlockDeviceImpl** 实现了 lwext4 需要的 `ext4_blockdev_iface` 回调表——包含 `dev_open`、`dev_read`、`dev_write`、`dev_close`、`dev_seek` 五个函数。这些函数将 lwext4 的块操作翻译为 **Disk trait** 调用，使 lwext4 能透明使用 virtio-blk 设备。

双架构差异。RISC-V 下 virtio 设备通过 MMIO 地址直接访问；LoongArch 下某些 virtio 设备通过 PCI 总线连接，需要额外的 PCI 枚举步骤。

【建议检查的代码】os/src/drivers/virtio/, os/src/drivers/disk.rs

【建议准备的图】用户 I/O → VFS → ext4 → page cache → **Disk trait** → virtio-blk 调用链

10.3 网卡驱动

virtio-net 设备的初始化流程与 virtio-blk 类似，但需要两条 virtqueue（发送队列和接收队列）。**NetDevice trait** 抽象了网卡的通用接口：`poll()` 驱动数据收发，`transmit()` 发送数据包，`receive()` 读取已到达的数据。

数据收发与 `smoltcp` 协议栈的集成点在于：设备 `poll` 时将接收到的原始以太网帧注入 `smoltcp` 的 `Interface` 对象，`smoltcp` 内部完成 IP 重组、TCP 状态机处理和 `socket` 投递；发送方向则从 `smoltcp` 获取待发送帧并填入 `virtqueue`。

10.4 串口驱动

NS16550A UART 是 x86/ARM/RISC-V 平台通用的串口控制器。RespOS 通过 MMIO 地址映射访问 UART 的寄存器（THR/RBR 用于收发、LSR 用于状态查询）。串口是内核早期启动阶段唯一的调试输出通道——在 `console` 初始化完成前，所有 `println!` 通过 UART 逐字节输出。

10.5 设备树与设备发现

设备树（Device Tree）是 RISC-V 平台描述硬件拓扑的标准方式。它是一个树形数据结构，描述 CPU 核心数、内存基址和大小、外设 MMIO 地址等关键信息。RespOS 在启动时解析 FDT，提取物理内存范围和 `virtio` 设备的 MMIO 基址。LoongArch 2K1000 平台不使用设备树，改为板级固定配置——内存大小和外设地址在 `board/2k1000.rs` 中硬编码。

【建议检查的代码】架构相关的设备树解析代码，`os/src/arch/*/config/board.rs`

11. 项目参考、AI 使用与第三方代码

本章回答: RespOS 参考了哪些项目? AI 工具如何参与开发? 第三方代码的来源和许可证是什么?

11.1 项目参考与相关对比

【待人工填写】

【本节目的】说明 RespOS 参考了哪些项目、与相关内核的设计差异。

【建议写什么】

- 基础项目: 基于哪个 rCore-Tutorial 版本 / 哪个 chX 起点
- 参考的内核项目及借鉴内容:
 - 如参考了 Phoenix 的 VFS 设计、Titanix 的调度器思路等
 - 每一项都要说"参考了什么"和"我们改了什么、为什么不同"
- 与其他竞赛内核的性能/功能对比 (如有数据)

【建议检查的代码】Git 最早 commit、README.md 中的参考声明

【建议准备的表】参考项目—参考内容—本队改进—差异说明表

11.2 AI 工具使用说明

【待人工填写】

【本节目的】完整披露 AI 工具的使用范围、方式和人工复核边界。

【建议写什么】

- 使用的 AI 工具清单 (名称、版本/模型、使用时间段)
- 各工具的使用场景:
 - 代码生成 / 代码补全
 - Bug 定位与调试辅助
 - 文档撰写与整理
 - 方案讨论与技术调研
- 每个场景说明输入/输出类型和人工复核方式

【容易出现的问题】

- 不能把所有代码归因于 AI; 也不能隐瞒 AI 参与的测试、Debug 或文档整理
- AI 输出的代码必须说明谁验证了生命周期、锁和失败路径

11.3 AI 产生内容与人工复核案例

【待人工填写】

【本节目的】用具体案例说明 AI 建议如何被验证、修改或拒绝。

【建议写什么】

- 选 2-4 个典型案例
- 每个案例格式: AI 输出摘要 → 实际采用部分 → 人工修改内容 → 源码/测试证据 → 未采用原因
- 区分"AI 建议日期"与"代码合入日期"

【建议准备的表】建议→人工审查→代码变更→测试结果流程表

11.4 第三方代码与依赖

项目/资料	使用位置	使用方式	本队修改
lwext4_rust	vendor/lwext4_rust/	ext4 后端, 通过 C FFI 提供 ext4 读写能力	封装为 Ext4Inode/Ext4SuperBlock 接入 VFS; 实现内存分配函数对接内核堆; 增加孤儿文件 rename-to-hidden 机制
smoltcp	vendor/smoltcp/、os/src/net/	网络协议栈, 提供 TCP/UDP/IPv4/IPv6 协议实现	封装 SocketFile 接入 VFS; 适配 loopback 设备和 virtio-net 驱动
riscv crate	vendor/riscv/、arch	提供 RISC-V CSR 寄存器和汇编指令的 Rust 封装	作为 HAL 层的底层依赖, 未做大改动

【建议检查的代码】Cargo.toml, vendor/README.md, vendor 中的 LICENSE 文件

【待人工确认】第三方库的版本/commit、License 和准确来源

11.5 资料引用与许可证说明

【待人工填写】

【本节目的】为源码、文档、图和附录的引用提供统一格式。

【建议写什么】

- README、vendor license、初赛参考资料、QEMU/OpenSBI/比赛资料的引用方式
- 版本日期和获取地址
- 字体与图片的版权归属（如适用）

12. 总结与展望

本章回答：RespOS 的最终贡献是什么？当前限制有哪些？如何复现？未来方向是什么？

12.1 本队最终贡献摘要

【待人工填写——从前面各章提炼 3-6 条核心贡献，每条绑定代码路径和测试状态】

【建议写什么】

- 双架构 HAL 抽象层的完整实现 (RISC-V64 + LoongArch64)
- 统一进程/线程模型的 TCB 设计，含细粒度字段分类、SMP 多核支持、exec sibling quiescence 协议
- 完整虚拟地址空间模型 (MemorySet→VMA→PTE)，含 COW、lazy allocation、file-backed ELF、FDT 动态内存识别
- 类 Linux VFS 架构，含 dentry 缓存、页缓存、挂载子系统、ext4 适配
- 每条贡献绑定代码路径和测试证据

【建议准备的表】贡献—代码—证据—状态摘要表

12.2 当前限制与未完成项

内核的已知边界按模块列出：

模块	当前限制
进程管理	vfork 地址空间共享语义未完整实现（仅有同步语义）；非 leader exec 未支持；LA64 SMP 未 经过动态验证；动态链接器仍为 eager load
内存管理	无 swap 和页面回收机制；LA64 RAM 上限仍为静态配置；私有文件页 fault 的锁外 I/O 未完 成；MS_INVALIDATE 与 SIGBUS 语义不完整
文件系统	单 ext4 超块实例，无多设备支持；写密集场景下 NAMEI_MUTATION_LOCK 串行化；全路径存储 dentry 模型 rename 和缓存淘汰有额外开销；页缓存使用 Vec 而非物理页框，mmap 路径有多 余拷贝
信号系统	SA_RESTART 未实现；实时信号排队不完整（同种信号连续到达时合并）；Core dump 未实现
时钟模块	线程 CPU clock、TAI、wakeup alarm 未支持；times() 的用户/系统时间仅为近似记账
网络模块	TCP 重传/拥塞控制依赖 smoltcp 内部实现；物理网卡驱动未完成；select 系统调用未实现
IPC	消息队列和 semaphore 未实现；跨 shmat 的 futex key 未统一
测试	BuildStorm 完整 minibuild/compile 未通过；LA64 多核测试覆盖不足

【建议准备的表】已验证/部分验证/未实现/待验证四分类表

【容易出现的问题】不能用"理论上支持"填入已验证列；限制不是道歉段落，而是结果可解释性的一部分

12.3 测试结果与性能数据

【待人工填写——补充最终测试数据】

【建议写什么】

- LTP 等测例通过情况（按测试层级分组：basic / busybox / libc / LTP / CAgent / 自研 probe）
- SMP 并发压力测试结果（按 SMP=1/2/4/8、debug/release、256M/8G 分列）
- 关键性能对比数据（如与前版本或参考内核的对比）

【建议准备的表】测试层级—测例数—通过数—主要失败原因表

12.4 复现入口

【待人工填写——最终提交版本确认后补充】

【建议写什么】

- 构建命令：make all 或 make build-rv / make build-la
- 镜像准备：bash scripts/get_img.sh
- 运行命令：make rv / make la
- 日志路径：rv-output.txt / la-output.txt

12.5 未来工作

基于当前限制（§12.2），下一步工作的优先级排序：

1. BuildStorm 完整通过。当前最重要的未闭环问题，涉及 pipe 引用生命周期、wait/wakeup 竞争、exit 延迟回收的综合调试
2. LA64 多核验证。当前 LA64 仍走单核路径，需要完成 LA64 的 SMP 启动、IPI 和共享 MM 验证
3. 内存管理完善。swap / 页面回收 / 动态链接器 file-backed 加载
4. 性能优化。per-CPU run queue、锁竞争优化、页缓存零拷贝 mmap
5. 功能补全。SA_RESTART、消息队列、物理网卡驱动

【建议写什么】每项未来工作必须与 §12.2 的限制一一对应；区分"已计划"与"方向性建议"